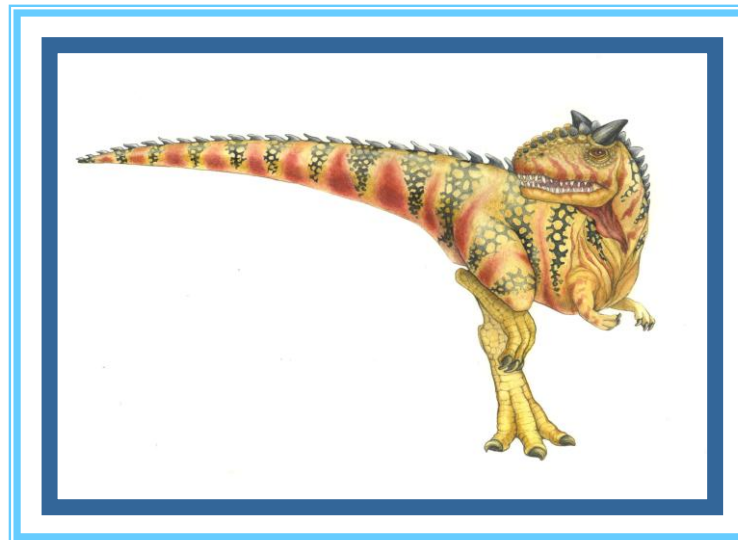


Unit-3: Deadlocks





The Deadlock Problem

- In the previous unit, the problem of deadlock was introduced during the discussion of **concurrent processes** and **their synchronization problems**. Deadlock is another **critical problem** in concurrent processes, and therefore, it must be known why it occurs in a system.
- There are some necessary conditions that give rise to a deadlock. If all of them hold true, the deadlock will certainly occur. If any one of the necessary conditions is prevented, the probability of deadlock becomes less. Some deadlock prevention methods have been proposed, but they are not always applicable.
- Deadlock avoidance is another approach to deal with deadlocks. For that, it requires to know the maximum resource requirement of the processes in advance, which is impractical. Therefore, this type of algorithms have low importance in practice.
- Deadlock detection is the method that a system must adopt if it is not possible to prevent or avoid the deadlock. Finally, once a deadlock has been detected in a system, recovery methods can be used.





DEFINING DEADLOCKS (Cont.)

- The computer system uses several types of resources, such as consumable or non-consumable, and preemptable or non-preemptable.
- In a multi-programming environment, the problem starts with **non-preemptable resources**. These resources have to be used in a mutually exclusive manner. But the problem is that resources are always limited, compared to the number of processes. In this case, when the concurrent processes request the resources, a deadlock occurs.
- Before defining a deadlock, let us understand how the resource allocation is done in a system. In general, the OS follows a protocol to use non-preemptive resources. The protocol consists of the following three events:
 - **Request:** Before using a resource, a process must request for it. This request is in the form of a **system call**, and it is passed on to the kernel. **This uses a data structure resource table to store the information about all the resources in the system.** Whenever the request is received, it checks the status of the resource. If the resource is free, the requesting process will get it. Otherwise, the requesting process will be blocked, and it must wait until the resource becomes free.





DEFINING DEADLOCKS (Cont.)

- **Use:** If a process gets the resource, the kernel changes the status of resource allocated and the **state of process to ready**, if it has been blocked earlier. **The process uses the resource in a mutually exclusive manner.**

- **Release:** After its use, a process releases the resource again through the use of a system call. The kernel can now allocate this resource to any blocked process or change the status of the resource to free.

- **The use of a non-preemptive resource may also follow the protocol of a critical section, which consists of the following criteria:**

- **Entry criteria:** The resource should be free while requesting, otherwise, the requesting process must wait.

- **Mutual exclusion:** The use of resource must be mutually exclusive.

- **Exit criteria:** There must be some criteria for releasing the resources.





DEFINING DEADLOCKS (Cont.)

- A semaphore can help the resources satisfy the above criteria. It will guard a resource and help in using it in a mutually exclusive manner (see Fig. 8.1).

```
Process A
{
semaphoresem_resource;

wait(sem_resource);
use the resource; //CS
signal(sem_resource);
}
```

Fig. 8.1 Using resource with semaphore





DEFINING DEADLOCKS (Cont.)

- Let us consider a system with two concurrent processes A and B, and with two resources such as one CD drive and one printer. Both the resources can be protected by using two semaphores (see Fig. 8.2).

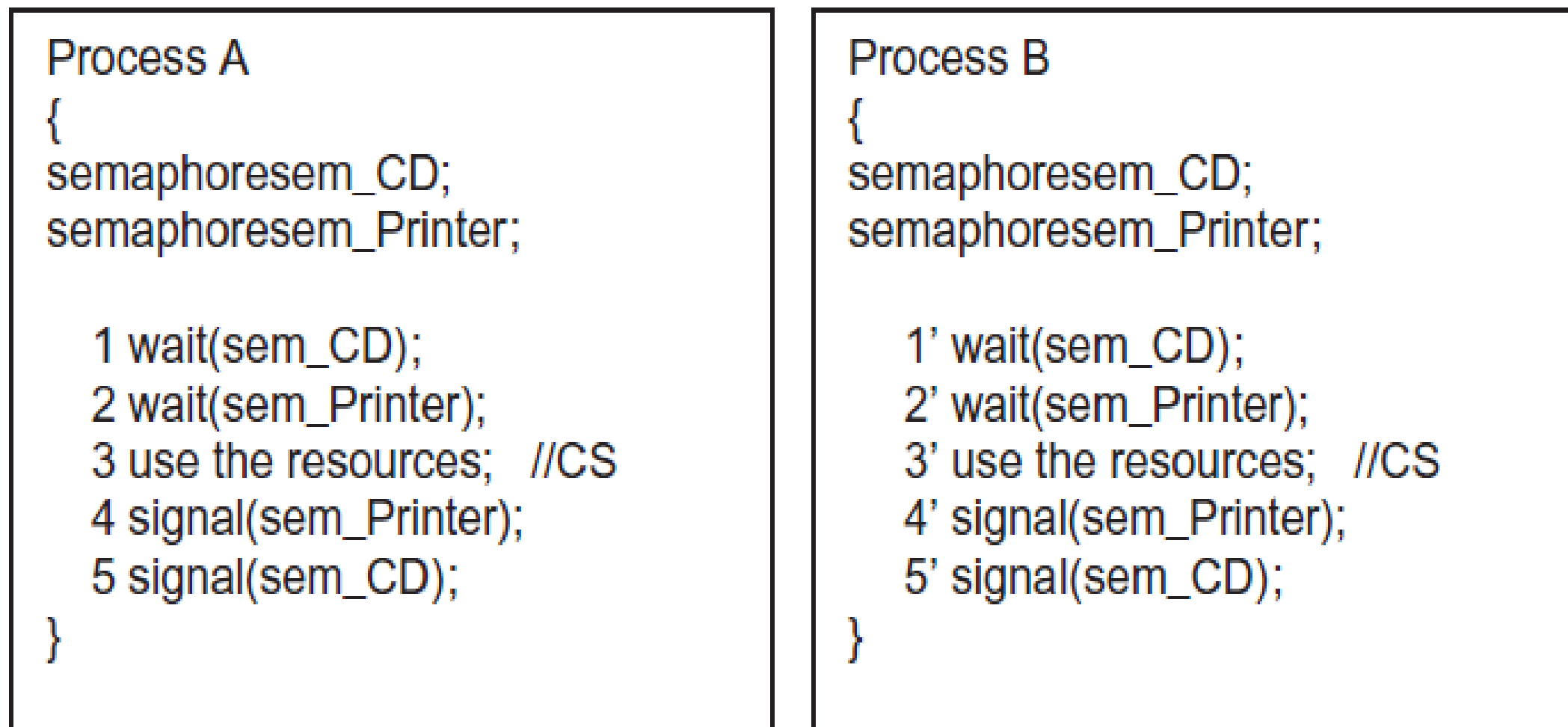


Fig. 8.2 Guarding two resources with two semaphores





DEFINING DEADLOCKS (Cont.)

- ❑ As we know, in a multi-programming environment, there may be interleaving of concurrent processes.
- ❑ In above example of Fig. 8.2, if process A starts first and holds the CD, then process B will not be able to start, as the CD is held by A. **Thus, A will use both resources without any interruption.** However, if there is a little change in the sequence of the instructions, the concurrent access of resources may not be as simple as shown in the Fig. 8.2.
- ❑ **It may be possible that B changes the sequence of requesting and releasing of the resources (see Fig. 8.3).** It first requests the printer, and then the CD. It releases the CD first, and then the printer. With this change in the code sequence, there will be conflicting demands of both processes. **In this scenario of concurrent processes, there may be interleaving of instructions as given in the following space: 1, 1', 2, 2'.**





DEFINING DEADLOCKS (Cont.)

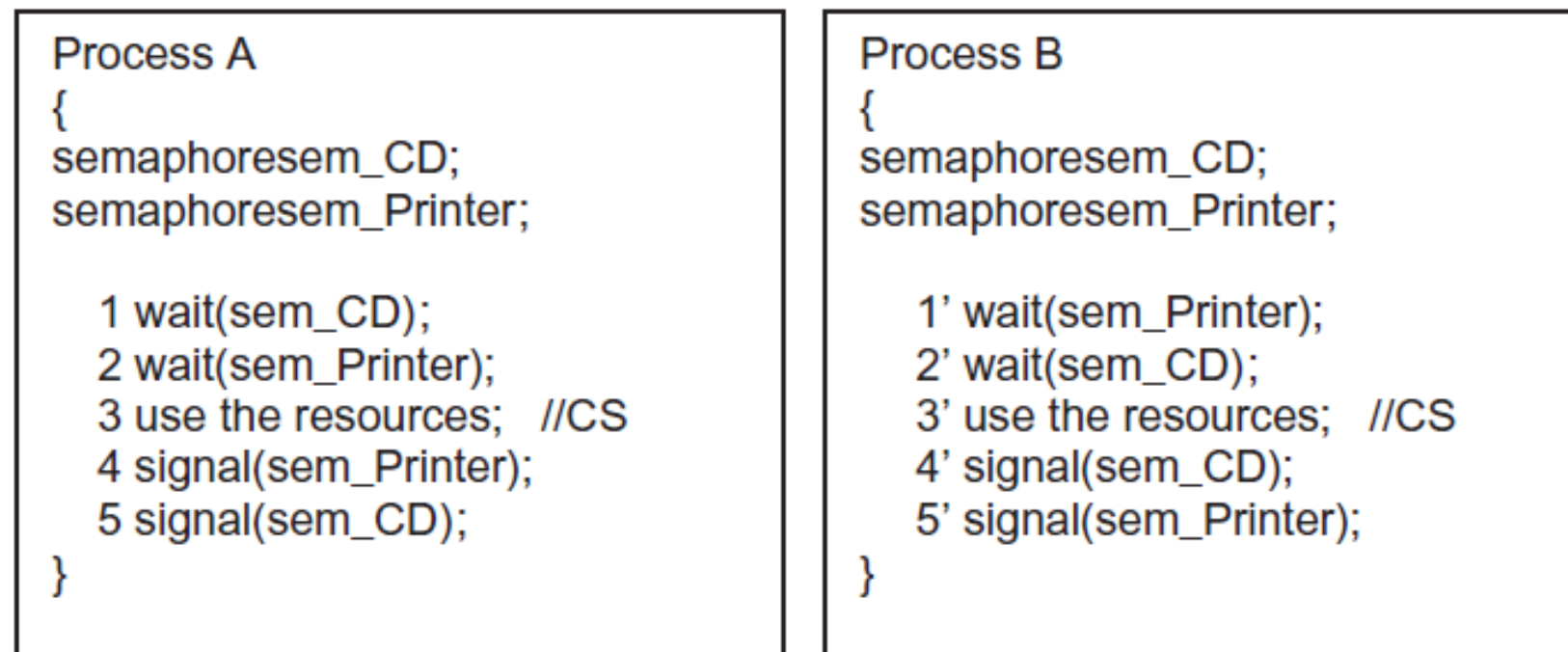


Fig. 8.3 Guarding two resources with two semaphores: Request and release sequence changed

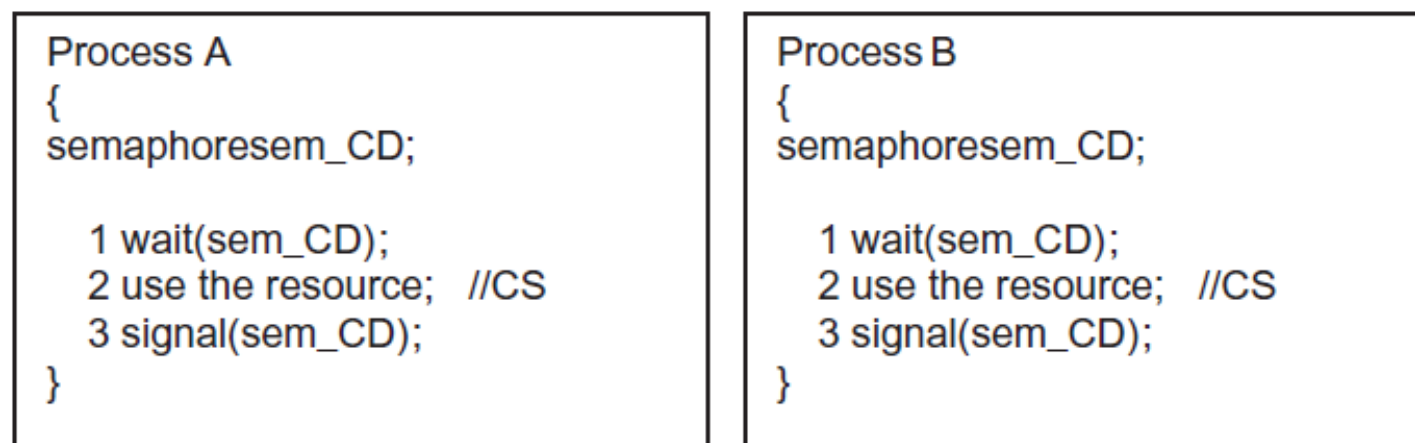
- Here, process A has held the CD, and process B has the printer. Then A requests for the printer and is blocked. Similarly, Process B requests for the CD and is blocked. **Thus, both processes hold one resource and are waiting for another resource held by the other process.** It can be realized here that the system will not progress because both processes wait for an infinite time. **This situation is known as a deadlock.**





DEFINING DEADLOCKS (Cont.)

- A deadlock can be defined as a situation when a set of concurrent processes in a system request for the resources in such a conflicting manner that there is an indefinite delay in **resource allocation**. In other words, every process in the system is blocked, and is waiting for the resources held by the other blocked processes. In this manner, no process is releasing any resource and cannot be awakened; therefore, the system is in a deadlock.
- Deadlocks may occur with non-preemptable consumable resources as well. The consumable resources, like messages once received by a process, are considered as consumed. Fig. 8.4 shows the semaphore as a consumable resource.





DEFINING DEADLOCKS (Cont.)

- Another example with messages causing deadlock is shown in Fig. 8.5.

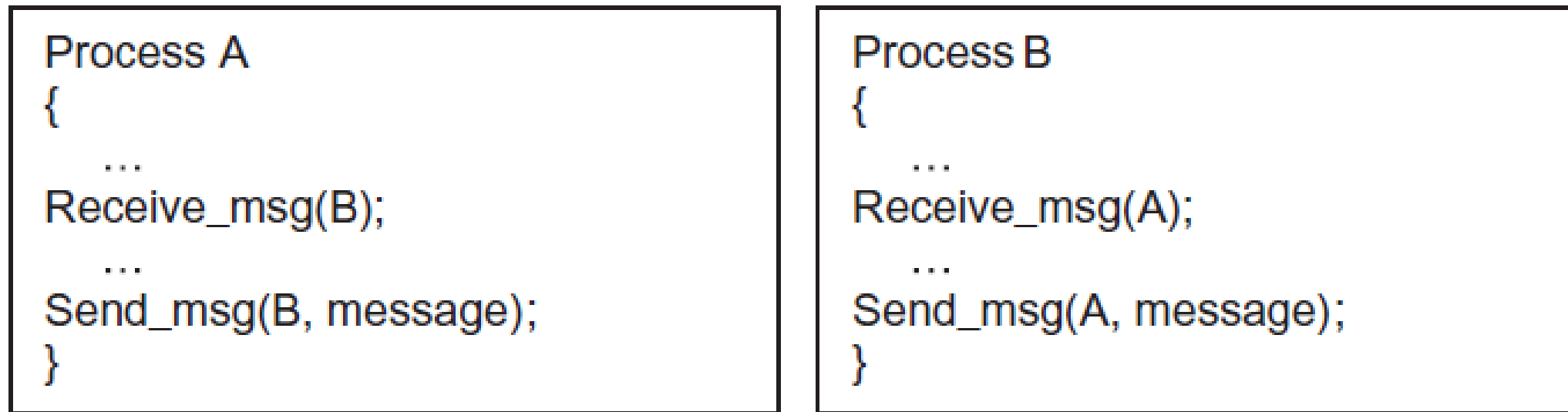


Fig. 8.5 Deadlock using Messages

- When a set of concurrent processes are in a deadlock situation, the degree of multiprogramming in the system decreases. This affects the performance of a system. In turn, it causes resource starvation because resources are tied up with deadlocked processes. Thus, deadlock situations are not favorable for a system. Moreover, deadlocks occur not only with user processes, but with OS processes as well. Thus, it becomes essential to identify and handle deadlock situations, whenever they occur, either with user processes/ system processes.





DEFINING DEADLOCKS (Cont.)

- It is obvious from the above examples that the problem is in the program, designed for the concurrent processes.
- Therefore, a careful approach for concurrent programming may avoid deadlocks. A proper ordering of allocation of resources to the processes may also help.
- **There are many ways of handling deadlocks. However, let us first discuss how to model a deadlock, and also the conditions that cause a deadlock in a system.**





MODELLING OF DEADLOCK

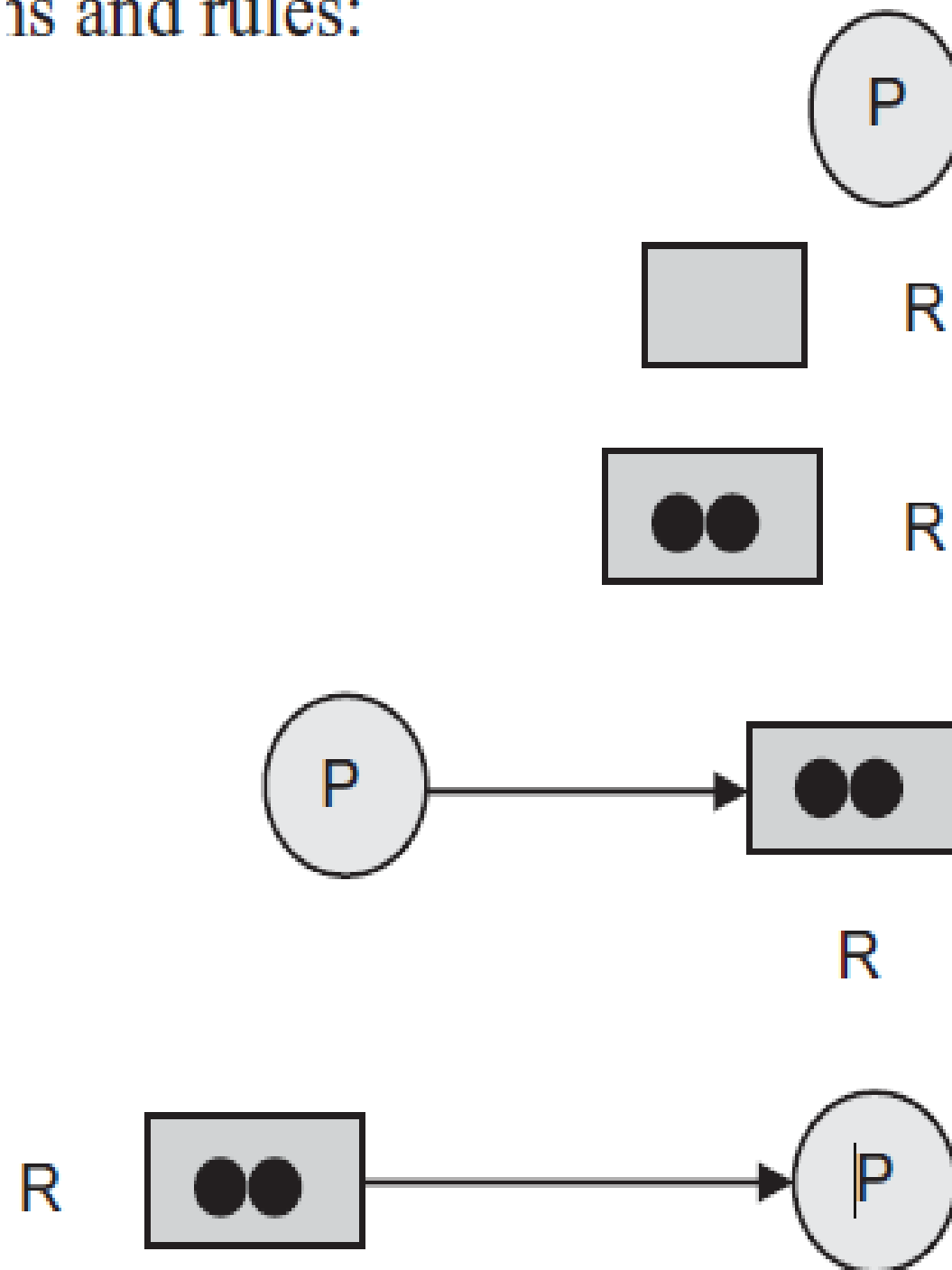
- The deadlock situation becomes complex when there are multiple processes. It is difficult to find out which process has caused the deadlock. **Therefore, a system needs to be represented using the processes and the state of resources in the form of a directed graph.** This directed graph, known as a **resource allocation graph (RAG)**, is a useful tool to represent the deadlock situation in a system.
- The RAG follows the following notations and rules:
- **The processes are represented as circular nodes.**
- **The resources are represented with a rectangle node.**
- The resource node contains some dots to represent the number of instances of that resource type. **The number of dots is equal to the number of instances.**
- An edge, from a process to a resource, indicates that the process has requested this resource but it has not been allocated. This is known as a **request edge**.
- An edge, from a resource node dot (resource instance) to a process, indicates that one instance of this resource type has been allocated to the process. This is known as an **assignment edge**.





MODELLING OF DEADLOCK (Cont.)

is and rules:





MODELLING OF DEADLOCK (Cont.)

- In Fig. 8.6 there are four processes and two resources. R1 has two dots, that is, there are two instances of R1. Similarly, R2 has a single instance. P1 has requested R1, and P3 has requested R2. One instance of R1 has been allocated to P2 and another one to P3. One instance of R2 has been allocated to P4.

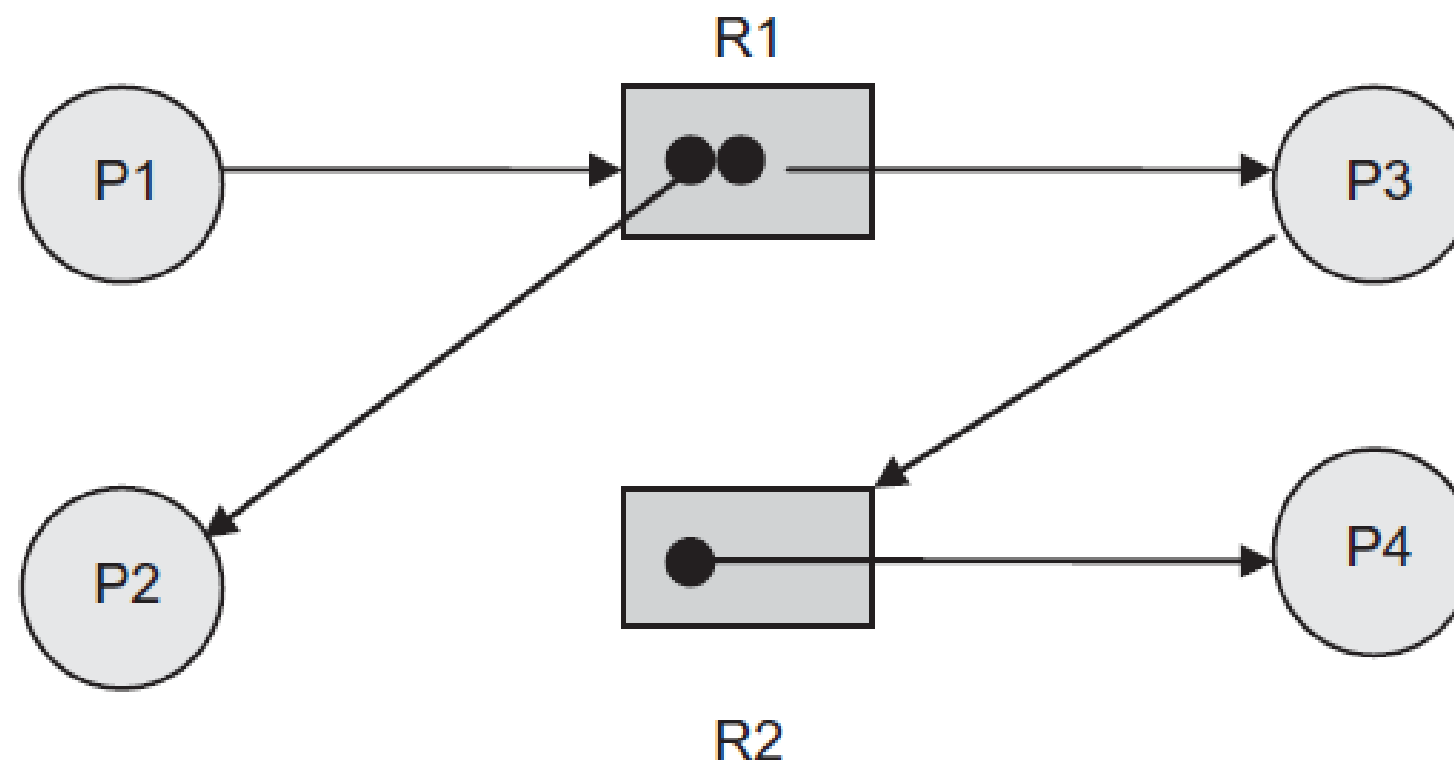


Fig. 8.6 Resource allocation graph





MODELLING OF DEADLOCK (Cont.)

- **Example**
- In a system, the following state of processes and resources are given:
- $P1 \rightarrow R1, P2 \rightarrow R3, R2 \rightarrow P1, R1 \rightarrow P3, P4 \rightarrow R3, R1 \rightarrow P4$
- Draw the RAG for the system.

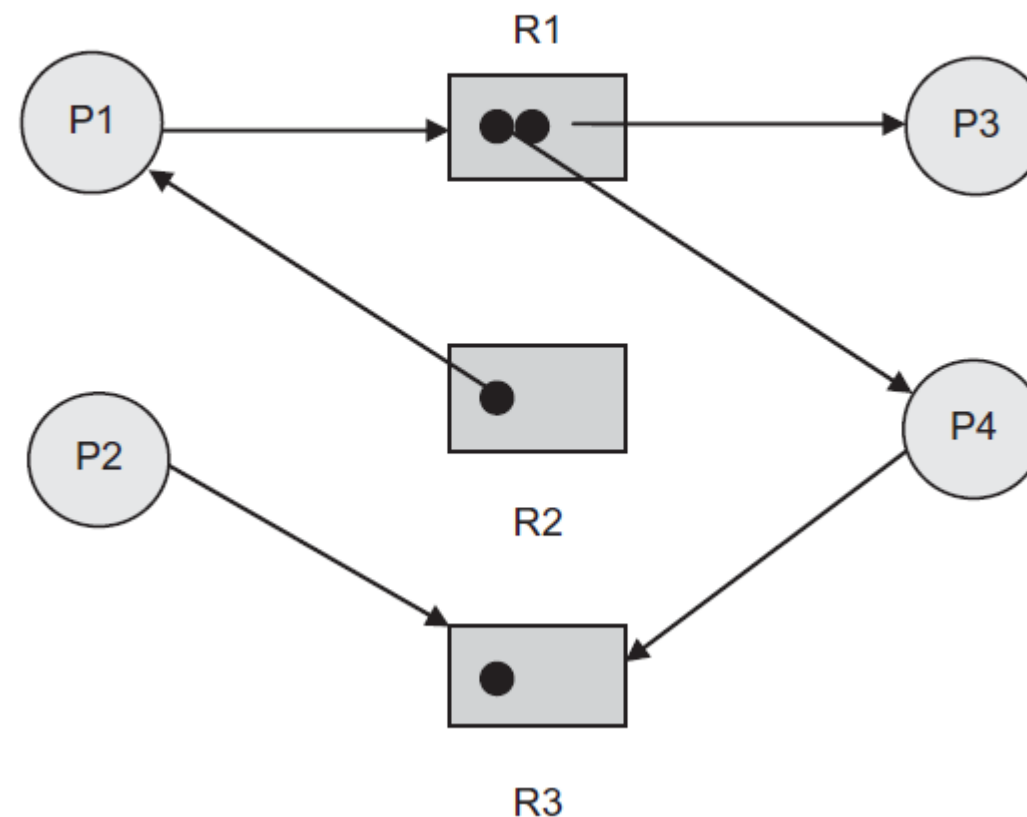




MODELLING OF DEADLOCK (Cont.)

- Example
- In a system, the following state of processes and resources are given:
- $P1 \rightarrow R1$, $P2 \rightarrow R3$, $R2 \rightarrow P1$, $R1 \rightarrow P3$, $P4 \rightarrow R3$, $R1 \rightarrow P4$
- Draw the RAG for the system.

Solution





CONDITIONS FOR DEADLOCK

- It is not necessary that a deadlock will always occur in concurrent process environments. **In fact, there are certain conditions that give rise to a deadlock.** The following are the conditions:
- **Mutual Exclusion:** The resources which require only mutually exclusive access, may give rise to a deadlock. These resource types cannot allow multiple processes to access it at the same time. **For example, a memory location, if allocated to a process, cannot be allowed to some other process. A printer cannot print the output of two processes at the same time. Therefore, a printer, if allocated to a process, cannot be allocated to another.**
- **Hold and Wait:** When all the processes are holding some resources and waiting for other resources, a deadlock may occur.
- **No Preemption:** The resources in a deadlock situation are of non-preemptable nature, as discussed above. If a resource cannot be pre-empted, it may lead to a deadlock.





CONDITIONS FOR DEADLOCK (Cont.)

- **Circular Wait:** These above three conditions together give rise to a fourth condition. A chain of processes may be produced, such that every process holds a resource needed by the next process. In this way, a circular chain of processes is built up in the system, due to mutually exclusive and non-preemptable nature of resources. The RAG shown in Fig. 8.7 depicts a deadlock.

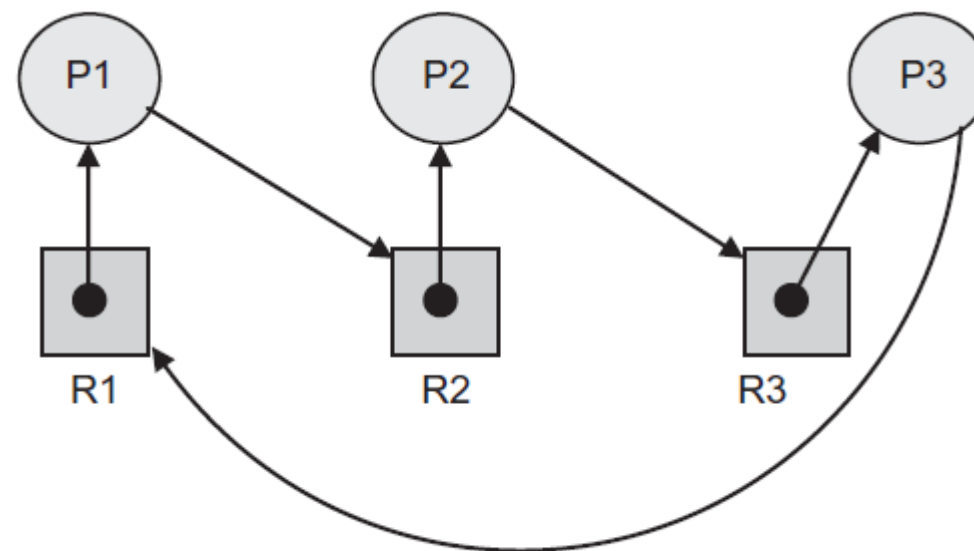


Fig. 8.7 RAG with deadlock

- In this way, the four conditions mentioned above, are necessary for a deadlock to occur. In fact, the policies inherent in the first three conditions cause the fourth condition, circular wait.





CONDITIONS FOR DEADLOCK (Cont.)

- **Example:** In a system, the following state of processes and resources are given:
- $R1 \rightarrow P1, P1 \rightarrow R2, P2 \rightarrow R3, R2 \rightarrow P2, R3 \rightarrow P3, P3 \rightarrow R4, P4 \rightarrow R3, R4 \rightarrow P4, P4 \rightarrow R1, R1 \rightarrow P5$
- **Draw the RAG for the system and check for deadlock condition.**

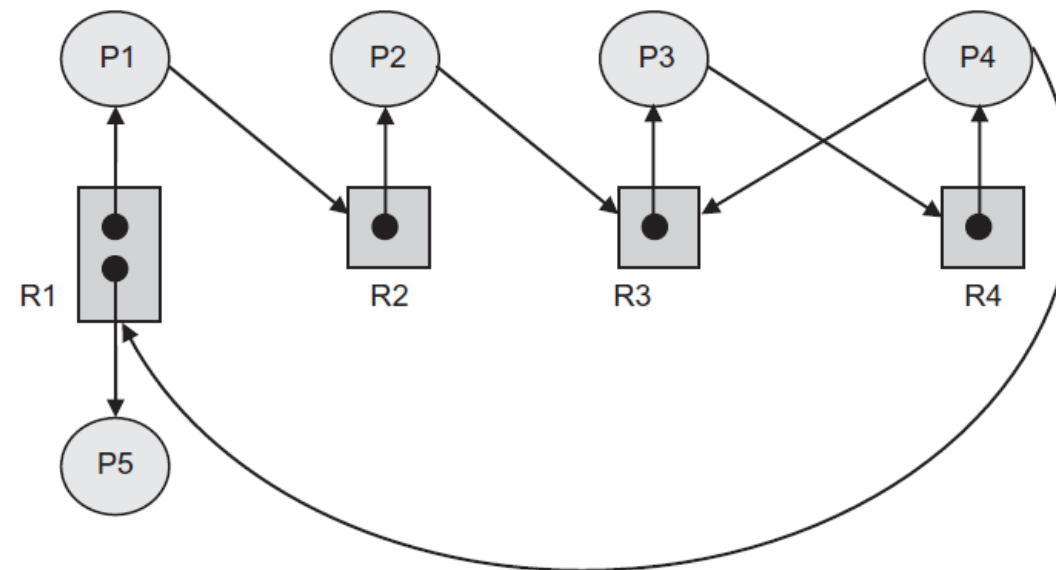




CONDITIONS FOR DEADLOCK (Cont.)

- **Example:** In a system, the following state of processes and resources are given:
- $R1 \rightarrow P1$, $P1 \rightarrow R2$, $P2 \rightarrow R3$, $R2 \rightarrow P2$, $R3 \rightarrow P3$, $P3 \rightarrow R4$, $P4 \rightarrow R3$, $R4 \rightarrow P4$, $P4 \rightarrow R1$, $R1 \rightarrow P5$
- Draw the RAG for the system and check for deadlock condition.

Solution



- In this scenario, processes P1, P2, P3, and P4 are holding one resource each, and requesting for one more resource, which is held by the next process. From the RAG drawn above, it can be seen that there is a cycle, in the form of circular wait in the graph, thereby, **causing a deadlock situation.**





CONDITIONS FOR DEADLOCK (Cont.)

- In shown above example, all the resource types have only a single instance, except R1. **There is a possibility that P5 may release the instance of R1, because it does not want any other resource type.** In that case, it will be allocated to P4. Consequently, the cycle will break and there will be no circular wait. **It means the circular wait is a necessary, but not a sufficient, condition for a deadlock.** If there are more than one instances of resource and there is a circular wait, there may or may not be a deadlock. **Moreover, if each resource type consists of only single instance, and there is a cycle in the graph, then the deadlock will occur.**
- In general,
- **Fact 1:** There are four necessary conditions for the occurrence of a deadlock: Mutual exclusion, hold and wait, non-preemption, and circular wait.
- **Fact 2:** The circular wait, along with the other three conditions, becomes the necessary condition for a deadlock, but not a sufficient, condition.
- **Fact 3:** The necessary and sufficient condition for a deadlock is that a cycle must be present in the RAG, that is, a circular wait is there, and all the resource types in the system have only one instance.





1. Example: DEADLOCK

Consider a system with four processes P_1, P_2, P_3, P_4 and four resources R_1, R_2, R_3, R_4 .

Each resource has **exactly one instance**. The following allocation and request situation is given:

- P_1 is holding R_1 and requesting R_2 .
- P_2 is holding R_2 and requesting R_3 .
- P_3 is holding R_3 and requesting R_4 .
- P_4 is holding R_4 and requesting R_1 .

1. Draw the Resource Allocation Graph (RAG).

2. Determine whether a deadlock exists. Justify your answer.





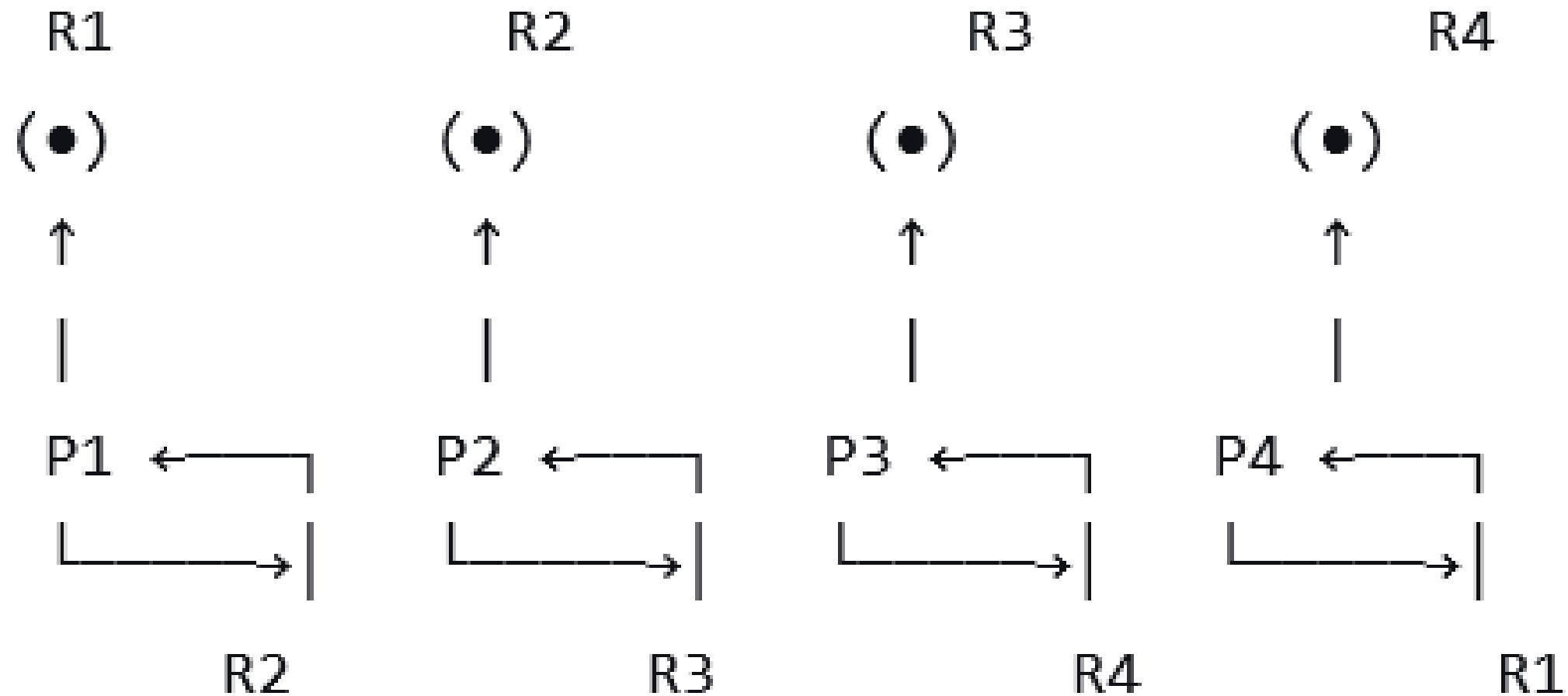
1. Example: DEADLOCK (Cont.)

- **Drawing the RAG**
- **Processes** are represented by circles: P1, P2, P3, P4.
- **Resources** are rectangles with one dot inside (single instance)
- **Assignment edge**: from a resource instance (dot) to a process.
- **Request edge**: from a process to a resource.
- The graph is:
- $P1 \leftarrow \text{assigned } R1$ and $P1 \rightarrow \text{requests } R2$
- $P2 \leftarrow \text{assigned } R2$ and $P2 \rightarrow \text{requests } R3$
- $P3 \leftarrow \text{assigned } R3$ and $P3 \rightarrow \text{requests } R4$
- $P4 \leftarrow \text{assigned } R4$ and $P4 \rightarrow \text{requests } R1$





1. Example: DEADLOCK (Cont.)





1. Example: DEADLOCK (Cont.)

□ Deadlock determination

- All resources are **single-instance**.

- The graph contains a **cycle**:

$$P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_4 \rightarrow P_4 \rightarrow R_5 \rightarrow P_5 \rightarrow R_1 \rightarrow P_1$$

- For single-instance resources, a **cycle** in the RAG is both **necessary and sufficient** for a deadlock.





2. Example: DEADLOCK (Cont.)

- Consider a system with three processes P1,P2,P3 and two resources R1,R2. Resource R1 has 2 instances. Resource R2 has 2 instances. **The current allocation and request edges are: P1 holds one instance of R1 and one instance of R2. It requests one more instance of R2. P2 holds one instance of R1 and one instance of R2. It requests one more instance of R1. P3 holds no resource and requests one instance of R1.**

1. Draw the RAG.
2. Is there a cycle? If yes, does the cycle necessarily imply a deadlock? Explain.
3. Determine whether the system is actually deadlocked. If not, show how a process can finish and release resources.





2. Example: DEADLOCK (Cont.)

□ 1. Drawing the RAG

- R1 has 2 instances: two dots in the rectangle. R2 has 2 instances: two dots in the rectangle.

□ Allocation edges:

- From a dot of R1 to P1
- From a dot of R1 to P2
- From a dot of R2 to P1
- From a dot of R2 to P2

□ Request edges: P1→R2(one instance)

- P2→R1 (one instance)
- P3→R1 (one instance)





2. Example: DEADLOCK (Cont.)

□ 2. Cycle analysis

- There is a cycle: $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$
- Because both R1 and R2 have multiple instances, a cycle is necessary but not sufficient for deadlock.

□ 3. Deadlock determination

- We must check whether the cycle actually blocks the system.
- Available instances of each resource:
Total R1=2, allocated to P1 and P2 = 2 $\rightarrow$ 0 available.
- Total R2=2, allocated to P1 and P2 = 2 $\rightarrow$ 0 available.
Processes P1 and P2 are in the cycle; each holds 1 instance of both resources and needs 1 more of the other.





2. Example: DEADLOCK (Cont.)

- No available instance to satisfy either request at this moment.
- But process P3 is not in the cycle and is requesting R1. However, P3 cannot proceed because no R1 instance is free. So P3 is blocked.
- Now, can any process finish?
- Processes P1 and P2 each hold resources; they cannot finish because they are waiting for each other. However, no process holds all the resources it needs; both P1 and P2 need one more resource. Thus, no process can release resources. The system is deadlocked.





3. Example: DEADLOCK (Cont.)

- Consider a system with: Resources: R1: 2 instances, R2: 1 instance, R3: 2 instances. Processes: P1, P2, P3, P4. The current allocation and requests:
 - P1 holds one instance of R1 and requests one instance of R2. P2 holds one instance of R2 and one instance of R3, and requests one instance of R1. P3 holds one instance of R3 and requests one instance of R1. P4 holds one instance of R1 and requests nothing.
- 1. Draw the RAG.
- 2. Identify the cycle(s).
- 3. Determine whether a deadlock exists. If yes, which processes are deadlocked? If no, explain why not.





3. Example: DEADLOCK (Cont.)

□ 1. Drawing the RAG

- R1 (2 instances): one dot allocated to P1, one dot allocated to P4. R2 (1 instance): allocated to P2.
- R3 (2 instances): one dot allocated to P2, one dot allocated to P3

□ Request edges:

- P1→R2
- P2→R1
- P3→R1
- P4 has no request.





3. Example: DEADLOCK (Cont.)

R_1 (● ●)

/ \

↑ ↑

| |

P_1 P_4

| |

└──→ R_2

R_2 (●)

↑

|

|

P_2

|

(holds)

|

└──→ R_1

R_3 (● ●)

/ \

↑ ↑

| |

P_2 P_3

|

└──→ R_1

|

(holds)





3. Example: DEADLOCK (Cont.)

- 2. Cycle identification
- Look for a directed cycle in the graph.
 - $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1 \rightarrow$ cycle.
 - $P3 \rightarrow R1 \rightarrow P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow$ this also returns to $P3$? Let's trace:
 $P3 \rightarrow R1$ (one of the instances), $R1$ is held by $P1$ and $P4$. Which instance is allocated to which? The cycle must follow a specific resource instance. Since $R1$ has two instances, we need to consider them separately. If $P3$'s request edge points to the same instance of $R1$ that is held by $P1$, then $P3 \rightarrow R1 \rightarrow P1 \rightarrow R2 \rightarrow P2 \rightarrow R1$ —now at $R1$ again, but this is a **different instance** (the one held by $P2$'s request is to a free instance? Actually $P2$ is requesting $R1$, not holding it. In the assignment edge, $R1$ is held by $P1$ and $P4$. So $P2$'s request edge goes to $R1$ as a resource, not to a specific instance. The cycle may be ambiguous.

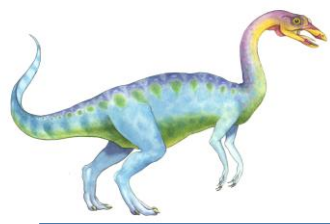




4. Example: DEADLOCK (Cont.)

- A system has four processes P1, P2, P3, P4 and three resources R1, R2, R3. Resource instances: R1: 2 instances, R2: 1 instance, R3: 2 instances
- The current allocation and request edges are:
 - P1 holds one instance of R1 and requests one instance of R2.
 - P2 holds one instance of R2 and requests one instance of R3.
 - P3 holds one instance of R3 and requests one instance of R1.
 - P4 holds one instance of R1 and requests no resource.
- 1. Draw the RAG.
- 2. Are there any cycles? If yes, list them.
- 3. Is the system deadlocked? Justify your answer.





4. Example: DEADLOCK (Cont.)

□ 1. Drawing the RAG

- **R1 has 2 instances: one allocated to P1, one to P4.**
- **R2 has 1 instance: allocated to P2.**
- **R3 has 2 instances: one allocated to P3; the other instance is free (not allocated).**

□ Request edges:

- **$P1 \rightarrow R2$**
- **$P2 \rightarrow R3$**
- **$P3 \rightarrow R1$**
- **P4 has no request.**





4. Example: DEADLOCK (Cont.)

R_1 (● ●)

/ \

↑

↑

|

|

P_1

P_4

|

└──→ R_2

R_2 (●)

↑

|

|

P_2

|

└──→ R_3

R_3 (● ●)

↑

|

|

P_3

|

└──→ R_1





4. Example: DEADLOCK (Cont.)

□ Directed edges:

- $P1 \rightarrow R2$, $P2 \rightarrow R3$, $P3 \rightarrow R1$, $R2 \rightarrow P2$ (assignment),
 $R3 \rightarrow P3$ (assignment)
- $R1 \rightarrow P1$ and $R1 \rightarrow P4$ (assignments)

□ 2. Look for a directed cycle:

- $P1 \rightarrow R2 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R1$ – from $R1$ we can go to $P1$ or $P4$.
If we go to $P1$, we close the cycle:
 $P1 \rightarrow R2 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R1 \rightarrow P1 \rightarrow$ **cycle exists.**
- If we go to $P4$, no edge out of $P4$ (no request), so no cycle.

- Thus, a **cycle exists**: $P1, P2, P3$ are involved in a cycle through the resources.





4. Example: DEADLOCK (Cont.)

- **3. Deadlock determination**
- Because R3 has **two instances**, but only one is allocated (to P3), there is **one free instance** of R3. **Process P2 is requesting R3; that request can be satisfied immediately from the free instance.**
- Once P2 receives the free R3 instance, it will eventually finish and release all its resources (R2 and the newly acquired R3). Those released resources will then satisfy P1 (needs R2) and possibly P3 (needs R1, but R1 is still held by P4, which is not waiting and will finish).
- **Hence, the cycle does NOT cause a deadlock because the free instance of R3 breaks the circular wait.**





5. Example: DEADLOCK (Cont.)

□ A system has five processes P1,P2,P3,P4,P5 and four resources R1,R2,R3,R4. Each resource has exactly one instance. The current state is given by the following edges (both allocation and request): P1 holds R1 and requests R2. P2 holds R2 and requests R3. P3 holds R3 and requests R1 and also R4. P4 holds R4 and requests R1. P5 holds no resource and requests R3.

1. Draw the RAG.

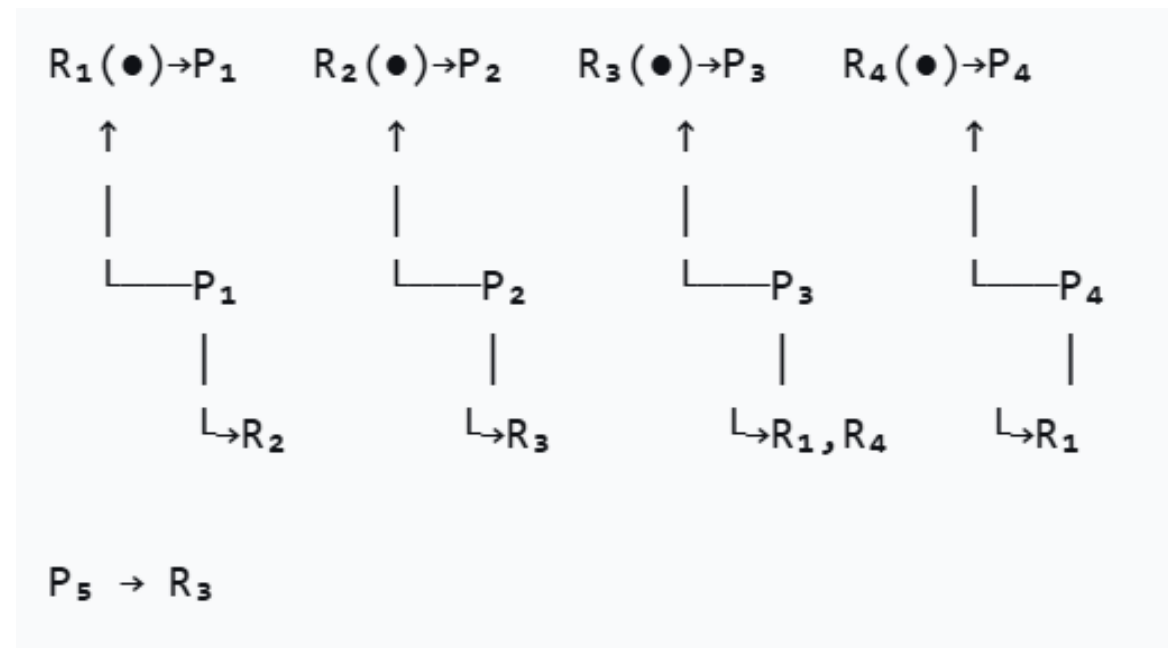
2. Identify all cycles in the RAG. Which processes are deadlocked?





5. Example: DEADLOCK (Cont.)

- 1. Drawing the RAG
- Each resource has a single instance.
- Assignment edges:
 - $R_1 \rightarrow P_1$, $R_2 \rightarrow P_2$, $R_3 \rightarrow P_3$, $R_4 \rightarrow P_4$
- Request edges:
 - $P_1 \rightarrow R_2$, $P_2 \rightarrow R_3$, $P_3 \rightarrow R_1$ and $P_3 \rightarrow R_4$, $P_4 \rightarrow R_1$, $P_5 \rightarrow R_3$





DEALING WITH DEADLOCK

After characterizing the deadlocks, the question is how to deal with them. Every system may be prone to deadlocks in a multi-programming environment, with few dedicated resources. To deal with the deadlock, the following three approaches can be used:

Deadlock Prevention: This method is very idealistic. It is based on the fact that if any of the four necessary conditions is prevented, a deadlock will not occur. Although it is very difficult to achieve this kind of prevention, this method is better, in the sense that the conditions for deadlock will not be allowed to arise in the system. Moreover, a principle can be established that a discipline is maintained, while the processes request for the resources.

Deadlock Avoidance: Deadlock prevention is not always possible. Another method to deal with the deadlock is to avoid it. But avoidance is possible when there is complete knowledge in advance, about which resources will be requested and used by the processes. At any instant of time, information such as, how much resources are available, how many are allocated to the processes, and what the future requests are, must be known. With this information, a process may be delayed, if its future request may produce a deadlock. In this way, the deadlock may be avoided.





DEALING WITH DEADLOCK (Cont.)

- **Deadlock Detection and Recovery:** If a system does not employ either **prevention or avoidance methods**, then a deadlock may occur. In this case, it becomes necessary to detect the deadlock. At least, it must be known that a deadlock has been occurred in the system. The processes, due to which the deadlock has occurred, should also be known to take action.
- **We should develop a mechanism to detect the deadlock in the system and resolve it, that is, recover from the deadlock situation.**





DEADLOCK PREVENTION

- The deadlock prevention method is based on the philosophy-‘**prevention is better than cure**’. The advantage of this method is that, it avoids the cost of detecting and resolving deadlocks, because this method will make any of the four conditions false and hence, prevent the deadlock. Let us see one by one how any one of the conditions can be prevented:
- Preventing Mutual Exclusion Condition: **It would be wrong to think that the mutual exclusion condition can be prevented.** The reason for this is that, mutual exclusion is an inherent nature of a resource, which cannot be changed. If we have to use a resource among multiple concurrent processes, it has to be used in a mutually-exclusive manner. Otherwise, no process would be able to get the desired output. **So, it is impossible to prevent mutual exclusion, but we can recognize resources which are sharable that can help us to prevent the deadlock situation in the system.**
- For example, read only files do not require mutually-exclusive access; any process can read this type of file. **So, basically mutual exclusion cannot be prevented, but the idea here is to recognize and use sharable resources as much as possible.**





DEADLOCK PREVENTION (Cont.)

□ Preventing Hold and Wait Condition: To prevent the hold-and-wait condition, two protocols can be used. **The first protocol is that, instead of requesting a resource on a need basis, each process should request all its resources at one time, and in advance. No process will be allowed to execute until it gets all of its declared resources. In this way, there will not be any conflict, pertaining to the requesting of resources, and no hold-and-wait condition will be generated in the system. However, this protocol suffers from the following disadvantages:**

1. A process may be idle for a long time, waiting for all the resources. This may decrease the degree of multi-programming.
2. Resource utilization will be reduced, because all the resources are not used throughout the execution of the process. **For example, a process copies some data from the CD to the hard disk and then sends the same for printing. In this case, it needs three resources, that is, the CD drive, hard disk space, and the printer. But if this process gets all the resources altogether, then the printer will be idle most of the time, because it will be used only at the end of the execution.**





DEADLOCK PREVENTION (Cont.)

3. Some frequently-used resources will always be held up with a process, resulting in starvation of other processes.
- The second protocol is that a requesting process should not be holding any resource. If a process needs another resource, it must first release the resource, it is currently holding. **This protocol may reduce the drawbacks of the first protocol to some extent.**
 - Preventing No Preemption Condition: This condition is necessary to break because sometimes deadlock can be prevented easily by preempting the resources. **In this method, if a process, holding some resources already, wants additional resources that are not free, then it should not wait. Rather, it should preempt all its resources, so that a deadlock situation can be avoided.** After some time, when the resources are free, the process may request all its resources altogether and start the execution. **However, there is a disadvantage. Imagine a situation, where a process is using a printer and the printer needs to be preempted in between. The problem with this method is that when a process releases its resources, it may lose all its work.**





DEADLOCK PREVENTION (Cont.)

Thus, this prevention method should be used with care and with the following guidelines:

1. The preemption is based on the execution of the processes. Suppose there are two processes, A and B. Each of them is holding a resource and needs another resource held by the other. **This will give rise to a deadlock**, if we do not preempt any of the resources. If A is about to complete its execution, while B has just started, the resource of B should be preempted, such that A finishes its execution without a deadlock.

2. The preemption is based on the priority level of the processes. If process B is of higher priority, the resource of process A should be preempted, and B should be allowed to complete first.

This method is costlier, in the sense that a process loses all its work and needs to start again. However, it is better and applied in rare cases only.

□ Preventing Circular Wait: Since circular wait condition is the consequence of the other three conditions, it can be prevented, if any of these three conditions is prevented. The circular wait can be prevented independently also through resource-ranking or ordering.





DEADLOCK PREVENTION (Cont.)

- Resource ordering is a method, in which every resource type in the system is given a unique integer number as an identification.
- Whenever a process requests a resource, it is checked whether it holds any other resource. If it does, IDs of both resource types are compared. **If the ID of a requested resource is greater than the ID of the resource it holds, the request is valid. Otherwise, it is rejected. Thus, following this resource-ordering, the processes can request resources in an increasing order of their IDs.** We can represent resource-ordering as a one-to-one function, given in the following space: $F: R \rightarrow I$, where, R is the set of resource type and I is the set of natural numbers for IDs.
- This protocol implies that if a process wants to request a resource type with lower ID as compared to the ID of resource type it holds, it must release all its resources. **The protocol is good enough for preventing the circular wait condition, but at the same time, it is very difficult to implement it practically.** Most of the resources are non-preemptable, and if they are released in between, in order to acquire other resources, a process will lose its work. The process then needs to acquire these released resources again. **In this way, this protocol may lead to a degraded performance of the processes.**





DEADLOCK PREVENTION (Cont.)

□ Some guidelines that may help in implementing this protocol are:

1. The ordering of resource types should be done depending on its use in the system. For example, if a system is using a CD drive and a printer, the printer should be given a higher ID as compared to the CD drive, as it will be used at the end of the processing. **Thus, careful numbering of resource types may help.**

2. All processes should request the resources **with lower ID, in the beginning itself.**





DEADLOCK PREVENTION (Cont.)

Example

A system uses three types of resources: hard disk, CD drive, and printer. Each of the resource types are allocated an ID, as shown in Fig. 8.8, and processes can request the resources in an increasing order of their IDs, if they are already holding a resource.

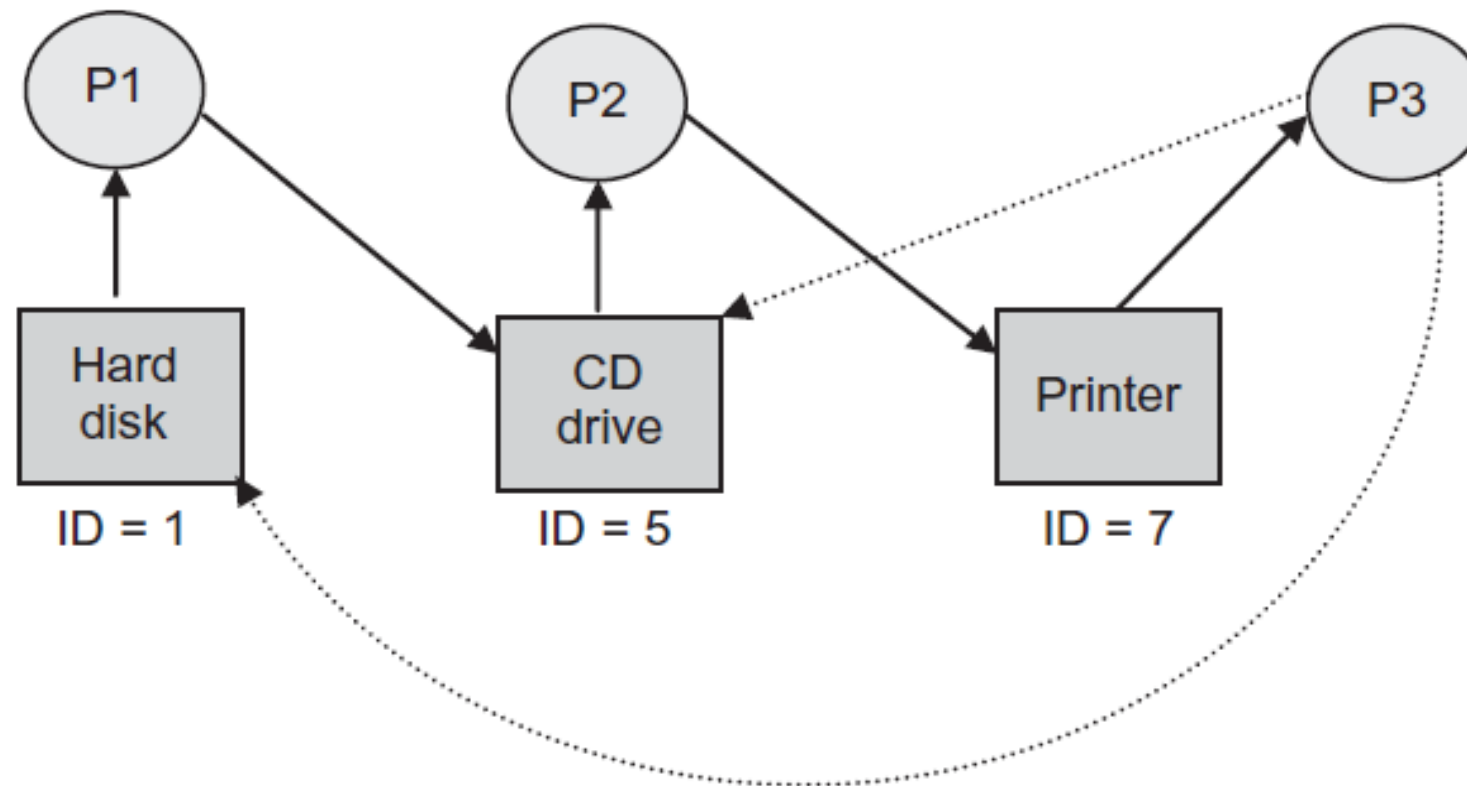


Fig. 8.8 Preventing circular wait





DEADLOCK PREVENTION (Cont.)

In this example, processes P1, P2, and P3 hold and request resources, as shown in the figure. If P3 requests the CD drive or hard disk, the request will be denied, as the ID of these resources is less than the ID of the printer. This will generate a cycle or circular wait condition in the system, thereby, producing a deadlock situation. The problem can be resolved by following the guidelines listed in previous slide 27. Thus, this example illustrates that circular wait can be prevented by allocating to resource types and IDs in an increasing order, with the condition that processes will request resources only in an increasing order of their IDs.

Deadlock prevention is used to prevent any of the conditions that cause deadlock. However, it results in degraded system efficiency and low device utilization. Therefore, this method cannot always be implemented.





DEADLOCK AVOIDANCE

- Another mechanism for avoiding deadlocks is **deadlock avoidance**, which checks in advance, the condition that may give rise to a deadlock. **It indicates the state of a system, such that if the request of a process for a resource gives rise to a deadlock condition, it is denied, and must wait.** In this way, the deadlock can be avoided. If the state of the system is such that it does not lead to a deadlock, then it is known as a safe state. **The converse, that is, deadlock state is known as an unsafe state.** The algorithm is run to check whether the requested resource changes the state of the system. If the resource will lead to an unsafe state, the request must wait, and if it does not affect the safe state, the request is granted.
- The deadlock avoidance algorithm, in the form of **safe state protocol**, must be run dynamically, whenever allocating a resource to a process. **In this way, deadlock avoidance approach is better than deadlock prevention, because it does not constrain the resources or processes, and there is no system performance degradation or device underutilization.**





DEADLOCK AVOIDANCE (Cont.)

The avoidance approach requires the knowledge of the maximum demand of all processes, all available resources, the presently allocated resources, and the future requests of the processes. **Whenever a resource request arrives, it is checked for its feasibility.** A resource request is feasible, only if the total number of allocated resources of a resource type does not exceed the total number of that resource type in the system. For example, say, the maximum available instances of a resource type are 4. **Suppose the current allocated resources are 3.** If a request for one resource arrives, it can be granted. However, if another request for the same resource type arrives, it exceeds the total available instances of that resource type, and therefore, this request must be rejected. **With the advance knowledge of system resources and process requests, it is possible to define a safe state,** which allows process requests without deadlock. However, if there is no possible safe state, the process is not allocated the resource, and it waits until there is a safe state in the system. **For example, in a system there are three processes P1, P2, and P3.** **There are three resources, the CD drive, the hard disk, and the printer.** P1 requires the CD drive and the printer. P2 requires the hard disk and the printer, and P3 requires all three resources. **If the sequence of resources and releases for each process is known, the system can be maintained in a safe state.**





DEADLOCK AVOIDANCE (Cont.)

❑ Example

- ❑ Consider a system with four processes that use resource R1, whose instances are 15. Each process declares its maximum demand of resources in advance, as shown in the following space. What will be the situation in the system at time instants t1, t2, and t3?

Process	Maximum demand	Demand at time instants		
		t1	t2	t3
P1	9	6	3	1
P2	4	2		
P3	6	3		
P4	8	2		

- ❑ **Solution:** At time instant t1, all processes can be allocated the resources, because each process does not exceed its maximum demand, and the sum of their demands is less than the total number of instances, that is, 15. Further, the system will be in a safe state, because the remaining resource demands of all the processes can be satisfied in a safe sequence P2, P1, P3, P4.





DEADLOCK AVOIDANCE (Cont.)

- ❑ At t_1 , after allocating the current demands, the remaining numbers of instances are 2. **So, only P2 can be satisfied at this moment.** However, after allocating 2 instances to P2, the process execution will be finished and it will release all 4 instances. **So we now have 4 instances, and P1 or P3 can be satisfied.** Similarly, the process goes on for other processes, and finally all the processes can be allocated instances of resource without any delay.
- ❑ **At time instant t_2 , there is a request of 3 more instances from P2.** But this will exceed the declared total number of maximum demand of P2. **So it is not a feasible request and hence, it would be rejected.**
- ❑ **At time instant t_3 ,** there is a request for one more instance from P3. This is a feasible request, but the remaining demands cannot be satisfied, and may lead to a deadlock. Since after allocating the demands of resources at t_2 , we have remaining one instance with which no process can be satisfied, the system may be in a deadlock state.





DEADLOCK AVOIDANCE (Cont.)

- This example illustrates the fact that some resource request may convert a safe state into an unsafe state. So, the idea is to check the state of the system, whenever a process requests for a resource. However, one point to be noted here is that an unsafe state does not always lead to a deadlock situation. In the above example, it may be possible that P2 does not demand its remaining resources, but releases all the resources it is holding currently. In this case, other processes may continue to execute, and there will be no deadlock. This happens only if the processes do not demand the maximum resources required and release their resources for the time being.
- Therefore, an unsafe state does not always cause a deadlock. But from the system's viewpoint, resources are not allocated to the processes that cause an unsafe state, and they need to wait until there is a safe state.





Deadlock Avoidance for Single Instance of Resources

□ To avoid a deadlock in a system, **where every resource type has a single instance of resource**, the RAG can be used again, but along with a new edge, known as **claim edge**. The claim edge is the same as request edge drawn from a process to a resource instance, but this does not mean that the request has been incorporated in the system. It is drawn in the dotted lines. The RAG can be used in such a way that when a process requests for a resource, a corresponding claim edge is drawn, and the graph is checked before converting it to a request edge.

□ That is, a process request will not be entertained until the **cycle check has been done**. After the cycle check, if it is confirmed that there will be no circular wait, the claim edge is converted to a request edge. Otherwise, it will be rejected. In this way, the deadlock is avoided.





Deadlock Avoidance for Single Instance of Resources

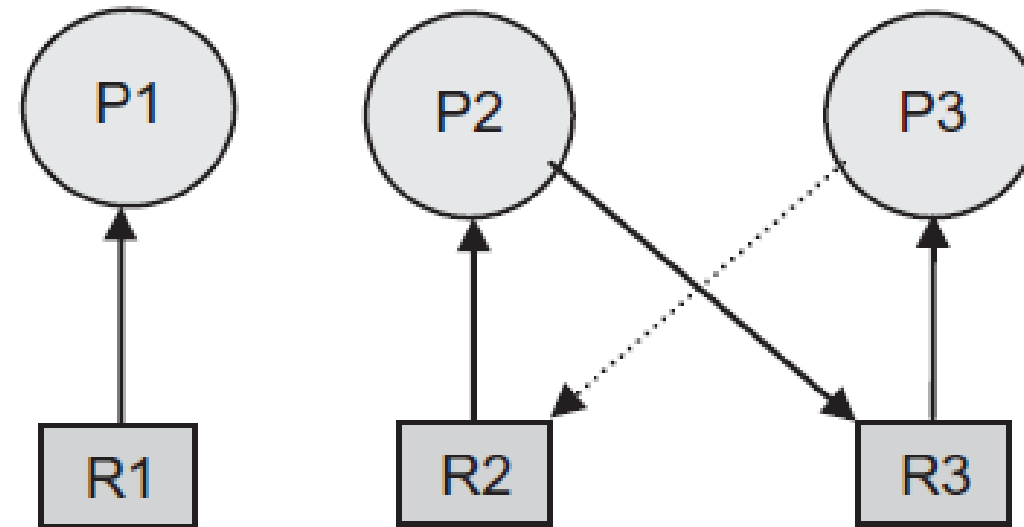
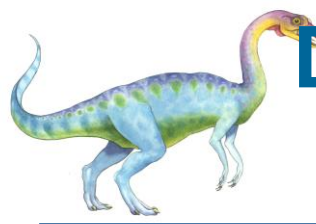


Fig. 8.9 RAG with claim edge

- In Fig. 8.9, it can be seen, that if there is a request from P3 for R2, a claim edge is drawn. However, it can be observed that if this request is considered, a cycle will be generated and deadlock will occur. Hence, the request will be denied.





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

□ The RAG-based cycle check cannot be applied when there are multiple instances of resources, because in this case, it is not for sure that deadlock will occur or not. Therefore, an algorithm is designed to check the safe state, whenever a resource is requested. **Dijkstra designed an algorithm, known as the Banker's Algorithm.** The algorithm takes analogy of a bank, where customers request to withdraw the cash. The banker has some data, based on which, cash is lent to a customer. The banker cannot give more cash than what a customer has requested for, and the total available cash. **Similarly, to design the algorithm for checking the deadlock, some data structures are maintained, such that whenever a resource is requested, it can be checked whether the request maintains the safe state of the system. If it does, the request can be granted; otherwise, the process must wait until there are sufficient resources available.**





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- **The banker's algorithm has two parts.** The first part is a **Safety Test algorithm** that checks the current state of the system for its safe state. The second part is **resource request-handling algorithm** that verifies whether the requested resources, when allocated to the process, affect the safe state. If it does, the request is denied. **In this way, banker's algorithm avoids the deadlock.**
- **Data Structures used:**
- **1. Total resources in a system**
- It stores the total number of resources in a system. Let us denote it as **Total_Res[i] = j**. **It means, there are j instances of resource type Ri in the system.** Thus, this is a vector of length r, that is, there are r numbers of resource types.





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- **2. Maximum demand of a process**
- Whenever a process enters the system, it declares its maximum demand of resources, which is stored in this data structure. Let us denote it as, $\text{Max}[i,j] = k$. **It means, process P_i has a total demand of k instances of resource type R_j .** Thus, this data structure is a $p \times r$ matrix, where p is the number of processes, and r is the number of resource types.
- **3. Current allocation of instances of each type of resource**
- It indicates the current allocation status of all resource types to various processes in the system. Let us denote it as, $\text{Alloc}[i,j] = k$. **It means, process P_i is allocated k instances of resource type R_j . Thus, this is also a $p \times r$ matrix.**





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- **4. Number of available resources**
- This data structure stores the current available instances of each resource type. Let us denote it as, $Av[i] = j$
- It means, j instances of resource type R_i are available. Thus, this is a vector of length r , that is, there are r numbers of resource types.
- This data structure is, in fact, the difference between the total resources and the allocated resources, that is, $Av[i] = Total_Res[i] - \sum \text{all processes } Alloc[i]$ where i is the resource type R_i .





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- ❑ **5. Current need of a process**
- ❑ This data structure indicates the current remaining resource need of each process. Let us denote it as, **$Need[i,j] = k$**
- ❑ It means, process P_i may require k more instances of resource type R_j , so that it can complete its execution. Thus, this is a $p \times r$ matrix. **This data structure is, in fact, the difference between the maximum demand of a process and the available resources, that is, $Need[i,j] = Max[i,j] - Alloc[i,j]$**
- ❑ **6. Request for a process**
- ❑ $Request_i$ is the vector to store the resource request for process P_i . Let us denote it as **$Req_i[j] = k$** . It means, process P_i has requested k instances of resource type R_j .





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

□ Safety Test Algorithm

□ Let **Current_Avail** and **Marked** be two vectors of length n and p , respectively. **Safe String** is an array to store the process IDs.

□ The algorithm is given by:

1. $Current_Avail = Av$
2. Initialize *Marked* as:
for($i=1; i \leq p; i++$)
 $Marked[i] = false$;
3. Find a process P_i such that
 $Need_i \leq Current_Avail$ and $Marked[i] = false$
4. if(found)
 {
 $Current_Avail = Current_Avail + Alloc_i$
 $Marked[i] = true$
 Save the process number in *SafeString*[]
 go to step 3.
 }
5. if($Marked[i] == true$) for all processes, then the system is in safe state.
 Print *SafeString*.
 Otherwise, the system is not in safe state, and is in deadlock.





Banker's Algorithm – Safety Test

- A system has five processes P0 through P4 and three resource types: A (10 instances), B (5 instances), C (7 instances).
- At time t0, the following snapshot is taken:

Process	Allocation (A, B, C)	Max (A, B, C)
P0	(0, 1, 0)	(7, 5, 3)
P1	(2, 0, 0)	(3, 2, 2)
P2	(3, 0, 2)	(9, 0, 2)
P3	(2, 1, 1)	(2, 2, 2)
P4	(0, 0, 2)	(4, 3, 3)

- Compute the Need matrix and the Available vector.
- Is the system in a safe state? If yes, give one safe sequence.





Banker's Algorithm – Safety Test...

□ Solution

□ 1. Compute Available vector:

□ Total resources: Total= (10, 5, 7)

□ **Allocated Sum** = (0+2+3+2+0, 1+0+0+1+0, 0+0+2+1+2) = (7, 2, 5)

□ **Available** = **Total** – **Sum (Allocation)** = (10–7, 5–2, 7–5) = (3, 3, 2)

□ **Compute Need matrix:**

□ **Need = Max–Allocation**





Banker's Algorithm – Safety Test...

Process	Allocation (A, B, C)	Max (A, B, C)	Process	Need (A, B, C)
P0	(0, 1, 0)	(7, 5, 3)	P0	(7, 4, 3)
P1	(2, 0, 0)	(3, 2, 2)	P1	(1, 2, 2)
P2	(3, 0, 2)	(9, 0, 2)	P2	(6, 0, 0)
P3	(2, 1, 1)	(2, 2, 2)	P3	(0, 1, 1)
P4	(0, 0, 2)	(4, 3, 3)	P4	(4, 3, 1)

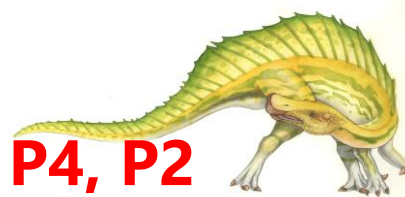




Banker's Algorithm – Safety Test...

□ 3. Safety Test: **Initial** Work=Available=(3,3,2); **Finish[i]=false for all i.**

- Step 1: Find i with Finish[i]=false and Needi ≤ Work.
P3: Need = (0,1,1) ≤ (3,3,2) ✓
Work = Work + Allocation = (3,3,2) + (2,1,1) = (5,4,3)
Finish[P3] = true; safe sequence = [P3]
- Step 2: Now check remaining processes.
P1: Need = (1,2,2) ≤ (5,4,3) ✓
Work = (5,4,3) + (2,0,0) = (7,4,3)
Finish[P1] = true; safe sequence = [P3, P1]
- Step 3: P0: Need = (7,4,3) ≤ (7,4,3) ✓
Work = (7,4,3) + (0,1,0) = (7,5,3)
Finish[P0] = true; safe sequence = [P3, P1, P0]
- Step 4: P4: Need = (4,3,1) ≤ (7,5,3) ✓
Work = (7,5,3) + (0,0,2) = (7,5,5)
Finish[P4] = true; safe sequence = [P3, P1, P0, P4]
- Step 5: P2: Need = (6,0,0) ≤ (7,5,5) ✓
Work = (7,5,5) + (3,0,2) = (10,5,7)
Finish[P2] = true; safe sequence = [P3, P1, P0, P4, P2]. All Finish=true.
- **Hence the system is in a safe state. One possible safe sequence: P3, P1, P0, P4, P2**





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

□ Resource Request Handling Algorithm

Let Req_i be the vector to store the resource request for process P_i .

if($Req_i Need_i$)

the request is not a feasible request and is rejected.

elseif ($Req_i > Av$)

Resources are not available, so the process must wait.

Otherwise,

{

Z that the requested resource has been allocated to the process and update the state as:

$Av = Av - Req_i$

$Alloc_i = Alloc_i + Req_i$

$Need_i = Need_i - Req_i$

Execute the *Safety Test Algorithm*, assuming the state has been changed.

if (state is safe)

Change the state in actual and the resource will be allocated to the process.

Otherwise,

Keep the state unchanged and do not allocate the resource to the process.

}





Banker's Algorithm – Resource Request...

- **Using the same system state as in Problem 2,** suppose at time t_1 process P_1 requests one additional instance of A and two instances of C:
- **Request $_1 = (1, 0, 2)$.**
- **Can the request be granted immediately?**
Apply the Banker's request handling algorithm.





Banker's Algorithm – Safety Test...

Process	Allocation (A, B, C)	Max (A, B, C)	Process	Need (A, B, C)
P0	(0, 1, 0)	(7, 5, 3)	P0	(7, 4, 3)
P1	(2, 0, 0)	(3, 2, 2)	P1	(1, 2, 2)
P2	(3, 0, 2)	(9, 0, 2)	P2	(6, 0, 0)
P3	(2, 1, 1)	(2, 2, 2)	P3	(0, 1, 1)
P4	(0, 0, 2)	(4, 3, 3)	P4	(4, 3, 1)





Banker's Algorithm – Resource Request...

- ❑ **Solution**
- ❑ **1. Check feasibility**
- ❑ • Request $\leq$ Need?
- ❑ Need = (1,2,2). Request = (1,0,2) $\leq$ (1,2,2) ✓
- ❑ • Request $\leq$ Available?
- ❑ Available = (3,3,2). Request = (1,0,2) $\leq$ (3,3,2) ✓
- ❑ **2. Pretend allocation**
- ❑ Update state as if request is granted:
- ❑ Available = (3,3,2) - (1,0,2) = (2,3,0)
- ❑ Allocation = (2,0,0) + (1,0,2) = (3,0,2)
- ❑ Need = (1,2,2) - (1,0,2) = (0,2,0)
- ❑ **Other processes are unchanged.**





Banker's Algorithm – Resource Request...

- ❑ 3. Run Safety Test on the new state
- ❑ **Work=Available=(2,3,0); Finish[i]=false.**
- ❑ • P1: Need = (0,2,0) ≤ (2,3,0) ✓
- ❑ Work = (2,3,0) + (3,0,2) = (5,3,2); Finish[P1] = true.
- ❑ • P3: Need = (0,1,1) ≤ (5,3,2) ✓
- ❑ Work = (5,3,2) + (2,1,1) = (7,4,3); Finish[P3] = true.
- ❑ • P0: Need = (7,4,3) ≤ (7,4,3) ✓
- ❑ Work = (7,4,3) + (0,1,0) = (7,5,3); Finish[P0] = true.
- ❑ • P4: Need = (4,3,1) ≤ (7,5,3) ✓
- ❑ Work = (7,5,3) + (0,0,2) = (7,5,5); Finish[P4] = true.
- ❑ • P2: Need = (6,0,0) ≤ (7,5,5) ✓
- ❑ Work = (7,5,5) + (3,0,2) = (10,5,7); Finish[P2] = true.

❑ All processes can finish. The state remains safe. Therefore, the





Banker's Algorithm – Problem

- A system has 5 processes (P0–P4) and 4 resource types (A, B, C, D). **The total resources are: A = 6, B = 7, C = 12, D = 10.**
- The allocation and max tables are:
- **Allocation:**

□ Process	A	B	C	D
□ P0	1	0	1	1
□ P1	2	1	0	2
□ P2	1	2	3	1
□ P3	0	3	1	2
□ P4	2	1	2	1





Banker's Algorithm – Problem...

□ **Max:**

□ Process	A	B	C	D
□ P0	2	1	2	3
□ P1	3	3	3	4
□ P2	2	4	6	3
□ P3	3	6	3	4
□ P4	4	3	4	3

□ **Available is initially (0, 0, 5, 3).**

□ **Determine if the system is in a safe state. If not, explain why.**





Banker's Algorithm – Problem...

□ **Solution:**

□ **Step 1: Compute Need = Max – Allocation.**

□ Process	A	B	C	D
□ P0	1	1	1	2
□ P1	1	2	3	2
□ P2	1	2	3	2
□ P3	3	3	2	2
□ P4	2	2	2	2

□ **Step 2: Work = Available = (0, 0, 3, 1). Finish = [false].**





Banker's Algorithm – Problem...

- Step 3: Try to find a process that can finish.
- • P0: Need $(1,1,1,2) \leq (0,0,3,1)$? No ($1 > 0$ in A and B).
- • P1: Need $(1,2,3,2) \leq (0,0,3,1)$? No (A and B fail).
- • P2: Need $(1,2,3,2) \leq (0,0,3,1)$? No.
- • P3: Need $(3,3,2,2) \leq (0,0,3,1)$? No (A, B fail).
- • P4: Need $(2,2,2,2) \leq (0,0,3,1)$? No (A, B fail).
- **No process can proceed. The system is unsafe because no process's Need is less than or equal to Available. Deadlock may occur if no resources are released. The system is in an unsafe state.**





Banker's Algorithm – Problem...

- A system has 5 processes (P0 through P4) and 4 resource types (A, B, C, D). The total resources are: **A=8, B=7, C=10, D=9**. The following snapshot of the system is taken at time T0:

Process	Allocation (A B C D)	Max (A B C D)
P0	0 0 1 2	0 0 1 2
P1	1 0 0 0	1 7 5 0
P2	1 3 5 4	2 3 5 6
P3	0 6 3 2	0 6 5 2
P4	0 0 1 4	0 6 5 6





Banker's Algorithm – Problem...

- **The Available vector is initially (1, 5, 2, 0).**
- 1. What is the content of the matrix Need?
- 2. Is the system in a safe state?
- 3. If a request from process P1 arrives for (0, 4, 2, 0), can the request be granted immediately?





Banker's Algorithm – Problem...

- Step 1: Compute the Need matrix. $\text{Need} = \text{Max} - \text{Allocation}$.

Process	Need (A B C D)	
P0	0 0 0 0	
P1	0 7 5 0	
P2	1 0 0 2	
P3	0 0 2 0	
P4	0 6 4 2	





Banker's Algorithm – Problem...

□ **Step 2: Check for a safe state.**

Initialize **Work = Available = (1, 5, 2, 0)** and **Finish = [false, false, false, false]**.

P0: Need (0,0,0,0) ≤ Work (1,5,2,0)? Yes (0≤1, 0≤5, 0≤2, 0≤0).

P0 can finish. Work = (1,5,2,0) + (0,0,1,2) = (1,5,3,2). Finish[0] = true.

P3: Need (0,0,2,0) ≤ Work (1,5,3,2)? Yes.

P3 can finish. Work = (1,5,3,2) + (0,6,3,2) = (1,11,6,4). Finish[3] = true.

P4: Need (0,6,4,2) ≤ Work (1,11,6,4)? Yes.

P4 can finish. Work = (1,11,6,4) + (0,0,1,4) = (1,11,7,8). Finish[4] = true.





Banker's Algorithm – Problem...

□ **Step 2: Check for a safe state.**

P2: Need (1,0,0,2) $\leq$ Work (1,11,7,8)? Yes.

P2 can finish. Work = (1,11,7,8) + (1,3,5,4) = (2,14,12,12). Finish[2] = true.

P1: Need (0,7,5,0) $\leq$ Work (2,14,12,12)? Yes.

P1 can finish. Work = (2,14,12,12) + (1,0,0,0) = (3,14,12,12). Finish[1] = true.

All processes finished $\rightarrow$ System is in a safe state.

A safe sequence is P0 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P2 $\rightarrow$ P1.





Banker's Algorithm – Problem...

- A system has a single resource type with 18 identical units. There are 4 processes (Job 1 to Job 4) with the following allocation and maximum requirements:

Process	Allocated	Maximum
Job 1	6	8
Job 2	2	9
Job 3	5	8
Job 4	3	5

- 1. How many available devices are there?
- 2. What are the remaining needs for each job?
- 3. Is the system in a safe state?





Banker's Algorithm – Problem...

- **Solution:**
- **Step 1: Calculate Available resources.**
- **Total resources = 18**
- **Total allocated = $6 + 2 + 5 + 3 = 16$**
- **Available = $18 - 16 = 2$ units.**
- **Step 2: Calculate remaining needs (Need).**
- **Need = Max – Allocation.**
- • **Job 1 Need = $8 - 6 = 2$**
- • **Job 2 Need = $9 - 2 = 7$**
- • **Job 3 Need = $8 - 5 = 3$**
- • **Job 4 Need = $5 - 3 = 2$**





Banker's Algorithm – Problem...

□ **Step 3: Check for a safe state.**

Work = Available = 2. Finish = [false, false, false, false].

Job 1: Need (2) $\leq$ Work (2)? Yes.

Job 1 can finish. Work = 2 + 6 = 8. Finish[0] = true.

Job 4: Need (2) $\leq$ Work (8)? Yes.

Job 4 can finish. Work = 8 + 3 = 11. Finish[3] = true.

Job 3: Need (3) $\leq$ Work (11)? Yes.

Job 3 can finish. Work = 11 + 5 = 16. Finish[2] = true.

Job 2: Need (7) $\leq$ Work (16)? Yes.

Job 2 can finish. Work = 16 + 2 = 18. Finish[1] = true.

All jobs can finish. The system is in a safe state. A safe sequence is Job 1 $\rightarrow$ Job 4 $\rightarrow$ Job 3 $\rightarrow$ Job 2.





Banker's Algorithm – Resource Request...

- Consider a system with 3 processes (P0, P1, P2) and 3 resource types (X, Y, Z). There are 3 units of X, 2 units of Y, and 2 units of Z available. The current allocation and max matrices are:

Process	Allocation (X Y Z)	Max (X Y Z)
P0	0 0 1	8 4 3
P1	3 2 0	6 2 0
P2	2 1 1	3 3 3





Banker's Algorithm – Resource Request...

- The system is currently in a safe state. Consider the following two independent requests:
- **REQ1: P0 requests (0, 0, 2).**
- **REQ2: P1 requests (2, 0, 0).**
- Which one of the following is TRUE?
- (A) Only REQ1 can be permitted.
- (B) Only REQ2 can be permitted.
- (C) Both REQ1 and REQ2 can be permitted.
- (D) Neither REQ1 nor REQ2 can be permitted.





Banker's Algorithm – Resource Request...

- Step 1: Analyze REQ1.
- Current Available = (3, 2, 2).
- Request REQ1 from P0 = (0, 0, 2).
- Check if **Request $\leq$ Need** of P0:
- Need of P0 = Max – Allocation = (8,4,3) – (0,0,1) = (8,4,2).
- **Request $\leq$ Need** (0,0,2) $\leq$ (8,4,2)? Yes.
- Check if Request $\leq$ Available: (0,0,2) $\leq$ (3,2,2)?
Yes (2 $\leq$ 2).





Banker's Algorithm – Resource Request...

- Pretend to allocate and update state:
- **New Available = $(3,2,2) - (0,0,2) = (3,2,0)$.**
- **New Allocation P0 = $(0,0,1) + (0,0,2) = (0,0,3)$.**
- **New Need P0 = $(8,4,2) - (0,0,2) = (8,4,0)$.**
- Check safety with Work = $(3,2,0)$:
- **P1: Need = Max – Allocation = $(6,2,0) - (3,2,0) = (3,0,0)$. Need $\leq$ Work $(3,2,0)$? Yes.**
- **Work = $(3,2,0) + (3,2,0) = (6,4,0)$. Finish P1.**
- **P2: Need = $(3,3,3) - (2,1,1) = (1,2,2)$. Need $(1,2,2) \leq$ Work $(6,4,0)$? No ($2 > 0$ in Z).**
- **P0: Need $(8,4,0) \leq$ Work $(6,4,0)$? No ($8 > 6$).**
- No further process can finish. The system becomes unsafe.





Banker's Algorithm – Resource Request...

- ❑ **Step 2: Analyze REQ2.**
- ❑ Current Available = (3, 2, 2).
- ❑ Request REQ2 from P1 = (2, 0, 0).
- ❑ Check if Request $\leq$ Need of P1: Need of P1 = (3,0,0). (2,0,0) $\leq$ (3,0,0)? Yes.
- ❑ Check if Request $\leq$ Available: (2,0,0) $\leq$ (3,2,2)? Yes.
- ❑ Pretend to allocate and update state:
- ❑ New Available = (3,2,2) – (2,0,0) = (1,2,2).
- ❑ New Allocation P1 = (3,2,0) + (2,0,0) = (5,2,0).
- ❑ New Need P1 = (3,0,0) – (2,0,0) = (1,0,0).
- ❑ Check safety with Work = (1,2,2):
- ❑ P1: Need (1,0,0) $\leq$ Work (1,2,2)? Yes.
- ❑ **Work = (1,2,2) + (5,2,0) = (6,4,2). Finish P1.**





Banker's Algorithm – Resource Request...

- P2: Need $(1,2,2) \leq$ Work $(6,4,2)$? Yes.
- Work = $(6,4,2) + (2,1,1) = (8,5,3)$. Finish P2.
- P0: Need $(8,4,2) \leq$ Work $(8,5,3)$? Yes.
- Work = $(8,5,3) + (0,0,1) = (8,5,4)$. Finish P0.
- **All processes can finish. The system remains safe.**
- Therefore, REQ2 can be permitted.
- **The correct answer is (B) Only REQ2 can be permitted.**





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

□ Example

- Consider a system with the following information. Determine whether the system is in safe state.

Total_Res

R1	R2	R3
15	8	8

Process	Max			Alloc		
	R1	R2	R3	R1	R2	R3
P1	5	6	3	2	1	0
P2	8	5	6	3	2	3
P3	4	9	2	3	0	2
P4	7	4	3	3	2	0
P5	4	3	3	1	0	1

- With the information given above, let us find the available resources. The number of available resources is calculated by:
- $Av[i] = Total_Res[i] - \sum \text{all processes } Alloc[i]$** , where i is the resource type R_i . Hence, the available resources are: **$Av[i]$**





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

R1	R2	R3
3	3	2

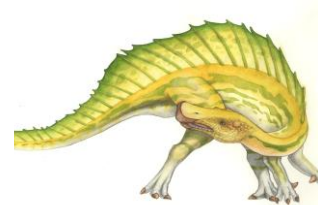
Next, let us find the *Need* matrix by subtracting *Alloc* from *Max* as given in the following space:

Process	Max			Alloc			Need(Max – Alloc)		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	5	6	3	2	1	0	3	5	3
P2	8	5	6	3	2	3	5	3	3
P3	4	8	2	3	0	2	1	9	0
P4	7	4	3	3	2	0	4	2	3
P5	4	3	3	1	0	1	3	3	2

Now let us execute the Safety Test algorithm to find whether the new state is safe.

Current_Avail = *Av* = [3 3 2]

Marked	
P1	false
P2	false
P3	false
P4	false
P5	false





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- The sequence of processes should be such that each process satisfies the criteria, $\text{Need} \leq \text{Current_Avail}$. A process, after finishing its execution, must release all the resources it is holding, so that the next process can avail them as per its need. The following table shows the process found, new value of **Current_Avail**, and **SafeString**. Repeat the procedure until all the processes complete their execution.

Process found	Current_Avail	SafeString []	Marked []
P5	$[3\ 3\ 2] + [1\ 0\ 1] = [4\ 3\ 3]$	P5	$\text{Marked}[P5] = \text{true}$
P4	$[4\ 3\ 3] + [3\ 2\ 0] = [7\ 5\ 3]$	P5, P4	$\text{Marked}[P4] = \text{true}$
P1	$[7\ 5\ 3] + [2\ 1\ 0] = [9\ 6\ 3]$	P5, P4, P1	$\text{Marked}[P1] = \text{true}$
P2	$[9\ 6\ 3] + [3\ 2\ 3] = [12\ 8\ 6]$	P5, P4, P1, P2	$\text{Marked}[P2] = \text{true}$
P3	$[12\ 8\ 6] + [3\ 0\ 2] = [15\ 8\ 8]$	P5, P4, P1, P2, P3	$\text{Marked}[P3] = \text{true}$

- From the table, it can be observed that the Marked value for all processes is true. Therefore, the system is in safe state. The Safe String of processes is {P5, P4, P1, P2, and P3}, that is, the processes can be scheduled in this order, to have a safe state in the system.





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- Consider the Example shown above and assume that the system is in safe state. At this moment, if P4 requests two more instances of R1 and two instances of R3, will the system still be in safe state? According to the requests made by P4, we have the request vector as:

$$\text{Req4} = [2 \ 0 \ 2]$$

- Now let us execute the resource request-handling algorithm, to find out whether the request can be granted. First check that Req4 is less than Need4. We find that $[2 \ 0 \ 2] < [4 \ 2 \ 3]$.
- Next, check that Req4 is less than Av. We find that $[2 \ 0 \ 2] \leq [3 \ 3 \ 2]$.
- So, we pretend that we allocate these resources and update the following: $\text{Av} = \text{Av} - \text{Req4} = [3 \ 3 \ 2] - [2 \ 0 \ 2] = [1 \ 3 \ 0]$

$$\text{Alloc4} = \text{Alloc4} + \text{Req4} = [3 \ 2 \ 0] + [2 \ 0 \ 2] = [5 \ 2 \ 2]$$

$$\text{Need4} = \text{Need4} - \text{Req4} = [4 \ 2 \ 3] - [2 \ 0 \ 2] = [2 \ 2 \ 1]$$





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- With the updated data structures, the state will be as given in the following space:

Av

R1	R2	R3
1	3	0

Process	Max			Alloc			Need		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	5	6	3	2	1	0	3	5	3
P2	8	5	6	3	2	3	5	3	3
P3	4	9	2	3	0	2	1	9	0
P4	7	4	3	5	2	2	4	2	3
P5	4	3	3	1	0	1	3	3	2

- Now let us check again whether the system is in a safe state.

Current_Avail = Av = [1 3 0]





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- There is no process whose **Need** $\leq$ **Current_Avail**. Therefore, no process can be started, and thus, this state would be unsafe, if the request for P4 is granted. The unsafe state may lead to deadlock, so the OS should not grant the resources requested by P4, until the system is in a safe state.
- **Deadlock avoidance is a wonderful algorithm, as it alerts the system in advance that the system is switching from safe state to unsafe state, and that there may be a deadlock situation.**
- **But is it really possible to implement this algorithm? The answer is no. One reason is that it is almost impossible to have the knowledge of maximum demand of each process in advance. Without this information, the algorithm cannot be implemented. However, with some analysis on a stable system, the maximum demand of processes for resources can be estimated, and therefore, the deadlock avoidance algorithm can be implemented.**





Dijkstra's Banker's Algorithm for Deadlock Avoidance in Multiple Instances of Resources

- Another reason is that in a multi-programming system, the number of processes is not fixed. In a time-sharing system, the processes are dynamic in nature; hence it may not be possible to have all the data structures required for deadlock avoidance algorithm.
- Another practical problem is the unavailability of resources at the required time. It is important that the resources must be available when a process requests for them, otherwise the algorithm cannot be implemented, even if the system knows in advance the total resources required by the process.
- Thus, the algorithm, due to these practical reasons, is rarely implemented. There is a lot of scope for researchers to investigate the problems inherent in this algorithm and make possible its implementation in order to avoid the deadlock.





DEADLOCK DETECTION

- Despite its strength to avoid the deadlock, deadlock avoidance algorithm cannot be implemented, due to practical reasons.
- **Thus, if a system is not able to implement neither deadlock prevention nor a deadlock avoidance algorithm, it may lead to a deadlock situation.**
- However, sometimes it is necessary to detect whether there is a deadlock in the system, and if there is, then which process is causing it. It is better to find the deadlock process, as it helps in recovering from the deadlock. The recovery process is the consequent action of deadlock detection, which will be discussed in the next section.
- **Deadlock detection algorithm also has two parts, such as avoidance, that is, detection of single instance of resource and detection for multiple instances of resources.**





DEADLOCK DETECTION (Cont.)

- Let us first discuss detection algorithm for single instance. But before discussing it, we need to have some modification in the RAG. **We will use RAG for detecting the cycle as done in avoidance algorithm.**
- **It becomes difficult to store the information of request edge and assignment edge, as it is required to store all these information, in order to make a RAG detect a cycle in it.**
- Therefore, in the graphical representation of RAG, some modification is done and a new graph, known as **wait-for graph**, is formed. In fact, RAG is optimized in **wait-for graph** by eliminating the resource nodes, and there are only edges between the processes.





DEADLOCK DETECTION (Cont.)

- In Fig. 8.11, the wait-for graph has been shown, corresponding to the RAG shown in Fig. 8.10. There is an edge from P1 to P2, because P1 waits for R1, and R1 is held by P2. Therefore an edge exists between the processes, only if one process waits for another.
- For example, an edge $P_a \rightarrow P_b$ exists in wait-for graph only if there are edges $P_a \rightarrow R_i$ and $R_i \rightarrow P_b$ in RAG.
- To detect the deadlock, it is checked whether the wait-for-graph contains a cycle. If it does, there is certainly a deadlock in the system (see Fig. 8.11). In this way, this cycle detection can be done periodically or when required to detect a deadlock in the system.

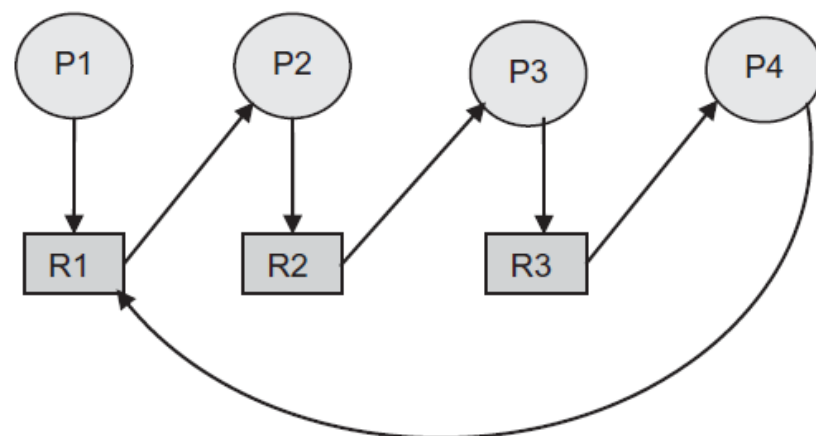


Fig. 8.10 RAG example

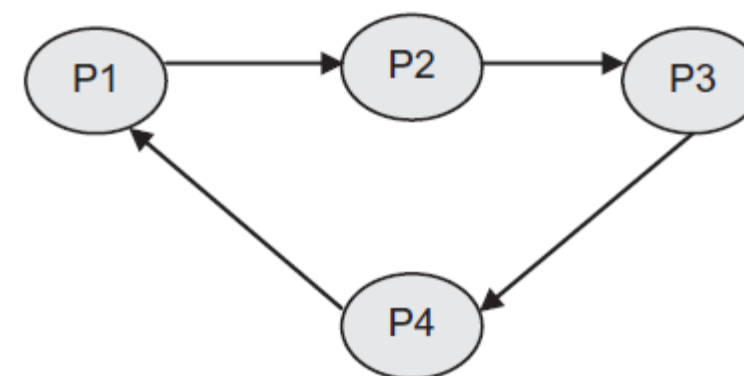


Fig. 8.11 Wait-for graph for Fig. 8.10





Deadlock Detection in Multiple Instances of Resource Types

- When there are multiple instances of resource types, then cycle detection is not sufficient to detect deadlock in a system. An algorithm, similar to the banker's algorithm, is designed. **This will check the system state with some data structures maintained and signal whether the system is in deadlock. The data structures used to detect deadlock are almost the same as in avoidance algorithm. The following data structures are used:**

1. *Total resources in system*

It stores the total number of resources in the system. $Total_Res[i] = j$

2. *Current allocation of instances of each type of resource*

It indicates the current allocation status of all the resource types to various processes in the system.

$$Alloc[i, j] = k$$

3. *Number of available resources*

This data structure stores the current available instances of each resource type.

$$Av[i] = j$$

$$Av[i] = Total_Res[i] - \sum_{all\ processes} Alloc[i], \text{ where } i \text{ is the resource type } R_i.$$

4. *Request of each process*

This data structure stores the current request of each process in the form of $p \times r$ matrix, where p is the number of processes and r is the number of resource types. Let us denote it as

$$Req[i, j] = k$$

It means, process P_i is requesting k instances of resource type R_j .



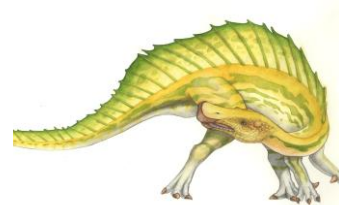


Deadlock Detection in Multiple Instances of Resource Types (Cont.)

Detection Algorithm

Let *Current_Avail* and *Marked* be two vectors of length n and p respectively. *Safe String* is an array to store the process IDs.

1. $Current_Avail = Av$
2. For all $i = 1$ to p ,
Initialize *Marked* as $Marked[i] = false$;
If ($Alloc_i = 0$)
Initialize *Marked* as $Marked[i] = false$;
3. Find a process P_i such that
 $Req_i \leq Current_Avail$ and $Marked[i] = false$
4. if (found)
{
 $Current_Avail = Current_Avail + Alloc_i$
 $Marked[i] = true$
Save the process number in *Safestring*[]
go to step 3.
}
5. if ($Marked[i] == true$) is for all processes, then the system is not in deadlocked state.
Print *Safe String*.
Otherwise, the system is in deadlock caused by Process P_i .





Deadlock Detection in Multiple Instances of Resource Types (Cont.)

□ Example

- Consider a system with the following information. Determine whether the system is in deadlock situation.

Total_Res

R1	R2	R3
5	6	4

Process	Alloc			Req		
	R1	R2	R3	R1	R2	R3
P1	1	0	2	1	0	0
P2	1	1	0	4	0	2
P3	1	1	0	0	1	2
P4	0	2	1	2	1	0
P5	1	2	0	3	1	4

□

- The total number of available resources is calculated by: $Av[i] = Total_Res[i] - \sum_{all\ processes} Alloc[i]$, where i is the resource type R_i . Hence, the available resources are: $Av[i]$





Deadlock Detection in Multiple Instances of Resource Types (Cont.)

R1	R2	R3
1	0	1

Now let us execute the detection algorithm.

$Current_Avail = Av = [1\ 0\ 1]$

Marked	
P1	False
P2	False
P3	False
P4	False
P5	False

- The sequence of processes should be such that each process satisfies the criterion, **Need $\leq$ Current_Avail**. A process, after finishing its execution, must release all the resources it is holding, so that the next process can avail them as per its need. The following table shows the process found, new value of Current_Avail, and SafeString. Repeat the procedure until all the processes complete their execution.





Deadlock Detection in Multiple Instances of Resource Types (Cont.)

Process found	<i>Current_Avail</i>	<i>SafeString []</i>	<i>Marked []</i>
P1	$[1\ 0\ 1] + [1\ 0\ 2] = [2\ 0\ 3]$	P1	Marked [P1] = true
P3	$[2\ 0\ 3] + [1\ 1\ 0] = [3\ 1\ 3]$	P1, P3	Marked [P3] = true
P4	$[3\ 1\ 3] + [0\ 2\ 1] = [3\ 3\ 4]$	P1, P3, P4	Marked [P4] = true
P5	$[3\ 3\ 4] + [1\ 2\ 0] = [4\ 5\ 4]$	P1, P3, P4, P5	Marked [P5] = true
P2	$[4\ 5\ 4] + [1\ 1\ 0] = [5\ 6\ 4]$	P1, P3, P4, P5, P2	Marked [P2] = true

- From the table, it can be observed that Marked value for all the processes is true. Therefore, the system is not in a deadlock state. The SafeString of processes is {P1, P3, P4, P5, and P2}, that is, the processes can be scheduled in this order, and therefore, there will be no deadlock.





RECOVERY FROM DEADLOCK

- The consequent action of deadlock detection is recovery from it. The aim of deadlock detection is to find out which process has caused the deadlock and resolve it, so that the system can resume its work. Imagine two cars approaching each other on a narrow bridge, through which only one car can pass at a time. If no car is ready to move back, then of course there will be deadlock situation forever. **This is same as in a no pre-emption condition.** So the only solution for resolving the deadlock is for one of the cars to move back (rollback) and allow the other car to pass through the bridge, thereby resolving the deadlock. An OS handles the deadlock in the same way.
- **There should be preemption of resources from one process, so that the other process can continue and the deadlock is removed. Another solution is to abort the processes which cause the deadlock.**





RECOVERY FROM DEADLOCK (Cont.)

- There are various solutions, but each one should be adopted with a lot of care.
- **Rollback or aborting a process cannot be implemented always, as there is a cost incurred in each solution.** There are some factors which guide us to choose a particular recovery method. The following are some recovery methods:
- **Resource Preemption:** Resource pre-emption is one of the methods with which to break a deadlock. In this method, resources are pre-empted from a process, which has caused the deadlock. **These resources are then given to other processes, such that other processes execute their completion and in this way, the deadlock is removed.** In other words, no pre-emption condition for deadlock must be broken, wherever applicable.





RECOVERY FROM DEADLOCK (Cont.)

- ❑ **However, there are various issues while adopting this solution.** The first issue is to choose a process whose resources can be pre-empted and given to other processes. This decision is driven by the following cost factors:
- ❑ Number and type of resources a deadlocked process is holding If the number of resources is less compared to the requirement of other processes to break the deadlock, or if the process is not holding the desired type of resource, then those resources need not be pre-empted.
- ❑ **Execution span of a process:** The process with the longest execution span should be pre-empted first. It may be possible that execution of a process is near completion, but it is in deadlock, due to shortage of just one resource instance. In this case, it is not feasible to pre-empt the process. A process, whose execution has just started and requires many resources to complete, will be the right victim for preemption.





RECOVERY FROM DEADLOCK (Cont.)

- **Starvation of process:** Based on the factors above, it may be possible that the same process is always chosen for resource pre-emption, **resulting in a starvation situation**. Thus, it is important to ensure that the process will not starve. This can be done by fixing the number of times a process can be chosen as a victim.
- After resource pre-emption of a deadlocked process, what happens to this process? The process is not able to retain its position in execution, and cannot continue. This process either has to be rolled back or be aborted and restarted. The decision depends on the situation of the processes in the system. Both approaches involve cost. The abort solution involves the cost of execution of the process from the start.
- **Rollback involves the cost of maintaining the checkpoints in the system and rollback to a safe state through the checkpoint.**





RECOVERY FROM DEADLOCK (Cont.)

- ❑ **Rollback**
- ❑ **Checkpoints are also a good solution to recover from a deadlock, apart from using it in the resource pre-emption. The checkpoints save the state of a system at specified point in time.**
- ❑ The resource state is saved in a separate file, along with the memory image. When a deadlock is detected, the state is analyzed and the deadlock process is detected. Further, the needed resource, which can break the deadlock, is also detected. The process, holding that resource, is rolled back to a safe checkpoint. After allocating the resource to the desired process, the system continues, thereby, recovering from the deadlock. The process, which was rolled back, restarts from the checkpoint.
- ❑ **There is another advantage of using the roll back method. The processes can be rolled back to a safe checkpoint and restarted from that point, such that, there is no deadlock.**





RECOVERY FROM DEADLOCK (Cont.)

- ❑ **Abort the process**
- ❑ Another method to recover from the deadlock is to abort some process, such that there is no deadlock in the system. However, it is a costly solution in terms of process computation. Alternatively, one process can be aborted at a time. After aborting a process, run the deadlock detection algorithm to confirm whether the deadlock still exists. Like resource pre-emption, here too, a victim process is chosen. Of course, the victim process should be such that the system is the least affected and runs without deadlock. There may be many factors to decide the victim process, which are as follows:
 - ❑ **Number and type of resources a deadlocked process is holding**
 - ❑ If the number of resources is less compared to the requirement of other processes to break the deadlock, it would be a waste, if we abort this process. Similarly, if the desired type of resource is not held by the process to be killed, then there is no use to kill it. Thus, this method is dependent on the number and type of resources the deadlocked process is holding.





RECOVERY FROM DEADLOCK (Cont.)

- ❑ **Execution span of a process**
- ❑ If a process is just near completion, and is in deadlock, due to requirement of only one resource instance, it is not good to kill this process. A process, which has just started, or requires many resources to complete its execution, will be the right victim for termination. Thus, the execution span of a process may determine the victim process.
- ❑ **Process priority**
- ❑ If the process is of high priority, there is danger in aborting it.
- ❑ **Type of process**
- ❑ Aborting of interacting processes may affect the system's performance, as it will not be able to interact with the user. Hence, it is always better to choose batch processes as victim.





PRACTICAL APPROACH FOR DEADLOCK HANDLING

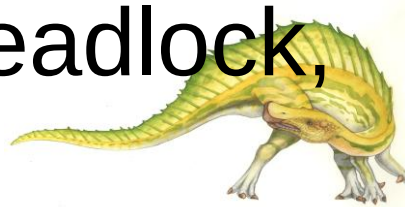
- All the methods, discussed above, can be employed in a system. However, the practical approach for handling deadlocks is to ignore it. Why?
- All the deadlock-handling strategies are costly to implement. The prevention methods, as discussed, are not applicable on every resource type. Only circular wait condition can be broken. The avoidance algorithm is also limited, because it is not possible to predict the maximum resources required by the processes in advance.
- The detection algorithm, employed in a system where there are too many processes, incurs cost. Thus a system, which is not mission-critical or business-critical, will ignore the deadlock, instead of employing any of the methods discussed.





PRACTICAL APPROACH FOR DEADLOCK HANDLING

- However, the deadlock-handling methods have their importance in real-time systems. Realtime systems, where the system must work continuously, cannot afford to ignore the deadlock.
- Like a PC or any other system, a real-time system cannot reboot or restart, because it may result in data loss. Therefore, all deadlock methods are employed in a real-time system with modifications.
- Distributed database systems and web systems also require deadlock-handling methods, because they handle a large number of records of various users and websites.
- No single method can be employed to handle the deadlock, instead an integrated strategy can be prepared for it.





PRACTICAL APPROACH FOR DEADLOCK HANDLING

- ❑ Some steps are given in the following space:
- ❑ Assign a resource to a process only when it is needed, that is, a process should not claim the resources in advance. This will reduce resources which need mutually-exclusive access.
- ❑ Mutual exclusion of some devices can also be handled with the help of creation of virtual devices. For example, spooling is used for making a virtual printer, and thereby, preventing the mutual exclusion condition.
- ❑ Apply the resource pre-emption, wherever possible. Some of the resources are preemptable by nature, for example, memory is a preemptable resource. In case of non-preemptable resources, depending on the nature of job of a process, there can be a forced pre-emption. For example, if a deadlocked process is using a printer, but the amount of job done by this process is very little, then we can pre-empt the printer forcibly and give it to some other process, so that the deadlock is broken.
- ❑ Always use a linear ordering of resource types, so that there is no circular wait, when processes request the resources. It is better to request the resources in an increasing order of their IDs, as discussed previously.
- ❑ **Deadlock detection is always a better choice in comparison with avoidance algorithm.**



End of Unit 3

